final_analysis

March 10, 2026

1 Homework 2 Quantitative Investment Management;

1.0.1 Team members - Ashish Puri, Roshan Patel and Nikhilesh Belulkar

2 Imports, data cleaning and exploratory data analysis

[5]:
```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
from datetime import datetime
```

[6]:
```
DF = pd.read_excel('SP500Raw.xlsx')
DF['date'] = pd.to_datetime(DF['date'])
DF = DF.set_index(keys=['date','permno']).copy()
DF
```

[6]:
```
                       price   shrout       prc          mcap
date       permno
2010-01-29 10104    23.06000  5011220 -0.057888  1.155587e+08
2010-02-26 10104    24.65000  5015000  0.068951  1.236198e+08
2010-03-31 10104    25.71000  5019091  0.043002  1.290408e+08
2010-04-30 10104    25.86750  5029523  0.008071  1.301012e+08
2010-05-28 10104    22.57000  5026000 -0.127477  1.134368e+08
…                       …       …        …            …
2024-08-30 93436   214.11000  3194640 -0.077391  6.840044e+08
2024-09-30 93436   261.63000  3207000  0.221942  8.390474e+08
2024-10-31 93436   249.85001  3210060 -0.045025  8.020335e+08
2024-11-29 93436   345.16000  3210060  0.381469  1.107984e+09
2024-12-31 93436   403.84000  3210060  0.170008  1.296351e+09

[90525 rows x 4 columns]
```

[7]:
```
DF.describe()
```

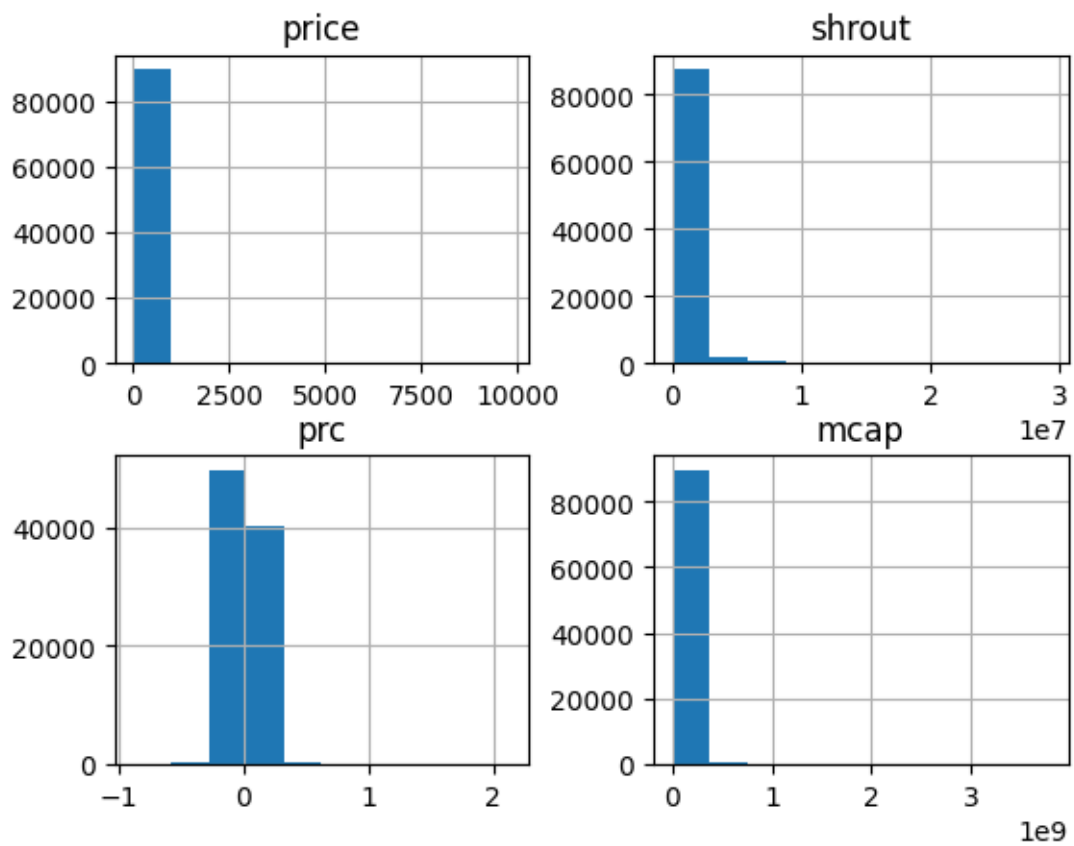
[7]:
```
              price        shrout           prc          mcap
count  90517.000000  9.052500e+04  90471.000000  9.051700e+04
mean     114.310440  6.208745e+05      0.011204  4.828252e+07
```

```
std      230.655848  1.191071e+06      0.085348  1.264082e+08
min        1.480000  3.064000e+03     -0.886269  6.203644e+05
25%       38.660000  1.584690e+05     -0.036248  1.059996e+07
50%       66.720000  3.002210e+05      0.011526  1.955465e+07
75%      118.360000  5.852470e+05      0.057079  4.227438e+07
max     9811.799800  2.920640e+07      2.135168  3.785304e+09
```

[8]:
```
plt.tight_layout()
DF.hist()
```

[8]:
```
array([[<Axes: title={'center': 'price'}>,
        <Axes: title={'center': 'shrout'}>],
       [<Axes: title={'center': 'prc'}>,
        <Axes: title={'center': 'mcap'}>]], dtype=object)
```

```
<Figure size 640x480 with 0 Axes>
```



[9]:
```
na_rows = DF[DF['price'].isna()]

for (date, permno), _ in na_rows.iterrows():
```

```
    all_prices_for_stock = DF.loc[(slice(None), permno), 'price']
    last_date_in_data = all_prices_for_stock.index.get_level_values('date').
↪max()
    prices_after_na = all_prices_for_stock.loc[date:].dropna()
    is_terminal = (date == last_date_in_data)

    if is_terminal:
        label = "TERMINAL - last row for this stock"
    elif len(prices_after_na) > 0:
        label = "MID-SERIES GAP - valid prices resume later"
    else:
        label = "TRAILING NA - no valid prices after this"

    print(f"permno={permno}  NA on {date.date()}  last date={last_date_in_data.
↪date()}  → {label}")
```

```
permno=14295  NA on 2022-10-31  last date=2022-10-31  → TERMINAL - last row for
this stock
permno=14704  NA on 2022-02-28  last date=2022-02-28  → TERMINAL - last row for
this stock
permno=25099  NA on 2015-06-30  last date=2015-06-30  → TERMINAL - last row for
this stock
permno=46877  NA on 2017-12-29  last date=2017-12-29  → TERMINAL - last row for
this stock
permno=82686  NA on 2022-09-30  last date=2022-09-30  → TERMINAL - last row for
this stock
permno=84020  NA on 2010-02-26  last date=2010-02-26  → TERMINAL - last row for
this stock
permno=88845  NA on 2018-11-30  last date=2018-11-30  → TERMINAL - last row for
this stock
permno=89014  NA on 2018-11-30  last date=2018-11-30  → TERMINAL - last row for
this stock
```

[10]:
```
na_prc_rows = DF[DF['prc'].isna()]

for (date, permno), row in na_prc_rows.iterrows():
    all_rows_for_stock = DF.loc[(slice(None), permno), :]
    dates = all_rows_for_stock.index.get_level_values('date')
    first_date = dates.min()
    last_date = dates.max()

    has_price_now = not pd.isna(row['price'])
    prior_dates = dates[dates < date]
    has_price_before = False
    if len(prior_dates) > 0:
        prev_date = prior_dates.max()
```

```
        has_price_before = not pd.isna(all_rows_for_stock.loc[(prev_date,␣
↪permno), 'price'])

    if date == first_date:
        reason = "FIRST OBSERVATION - no prior month to compute return"
    elif not has_price_now and not has_price_before:
        reason = "BOTH PRICES MISSING - no price at t or t-1"
    elif not has_price_now:
        reason = "PRICE MISSING AT t - can't compute return"
    elif not has_price_before:
        reason = "PRICE MISSING AT t-1 - can't compute return from prior month"
    elif date == last_date:
        reason = "LAST OBSERVATION - has price but prc still NA (data issue?)"
    else:
        reason = "UNKNOWN - has prices at t and t-1 but prc is NA"

    print(f"permno={permno}  date={date.date()}  price={'YES' if has_price_now␣
↪else 'NA'}  prev_price={'YES' if has_price_before else 'NA/NONE'}  →␣
↪{reason}")
```

```
permno=12456  date=2011-01-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=12872  date=2011-07-29  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13035  date=2011-11-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13141  date=2012-01-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13168  date=2011-12-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13356  date=2012-05-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13567  date=2012-10-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13586  date=2012-10-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13598  date=2012-10-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13721  date=2013-01-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=13963  date=2013-07-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=14295  date=2022-10-31  price=NA  prev_price=YES  → PRICE MISSING AT t -
can't compute return
permno=14297  date=2013-12-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=14542  date=2014-04-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
```

```
permno=14601  date=2014-05-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=14704  date=2022-02-28  price=NA  prev_price=YES  → PRICE MISSING AT t -
can't compute return
permno=15401  date=2015-07-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=15404  date=2015-07-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=15408  date=2015-07-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=15488  date=2015-07-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=15707  date=2015-11-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=15797  date=2015-11-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=15980  date=2016-04-29  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=16087  date=2016-07-29  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=16342  date=2016-10-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=16538  date=2017-01-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=16816  date=2017-08-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=16851  date=2017-09-29  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=18143  date=2018-10-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=18420  date=2019-03-29  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=18421  date=2019-03-29  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=18428  date=2019-04-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=18592  date=2019-06-28  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=18724  date=2019-06-28  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=19285  date=2020-04-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=19286  date=2020-04-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=19807  date=2020-10-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=20057  date=2020-11-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
```

```
permno=21124  date=2021-06-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=22623  date=2022-02-28  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=22976  date=2022-04-29  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=23570  date=2023-01-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=24174  date=2023-10-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=24877  date=2024-04-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=24878  date=2024-04-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=25099  date=2015-06-30  price=NA  prev_price=YES  → PRICE MISSING AT t -
can't compute return
permno=25146  date=2024-07-31  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=25434  date=2024-09-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
permno=46877  date=2017-12-29  price=NA  prev_price=YES  → PRICE MISSING AT t -
can't compute return
permno=82686  date=2022-09-30  price=NA  prev_price=YES  → PRICE MISSING AT t -
can't compute return
permno=84020  date=2010-02-26  price=NA  prev_price=YES  → PRICE MISSING AT t -
can't compute return
permno=88845  date=2018-11-30  price=NA  prev_price=YES  → PRICE MISSING AT t -
can't compute return
permno=89014  date=2018-11-30  price=NA  prev_price=YES  → PRICE MISSING AT t -
can't compute return
permno=93422  date=2010-07-30  price=YES  prev_price=NA/NONE  → FIRST
OBSERVATION - no prior month to compute return
```

[11]:
```
DF = DF.dropna(subset=['price'])
```

[12]:
```
DF.isna().sum()
```

[12]:
```
price      0
shrout     0
prc       46
mcap       0
dtype: int64
```

3 Why does the dataset use PERMNO and not TICKERS, which is how stocks are identified on the major exchanges in the US?

The dataset uses PERMNO (Permanent Number) rather than tickers because PERMNO is a stable, unique identifier that never changes for a given security, whereas tickers are unreliable as long-term identifiers for several reasons:

1. **Mergers and acquisitions.** A company may change its name and ticker after being acquired or merging with another company (e.g., Time Warner → WarnerMedia).
2. **Multiple share classes.** A single company can have multiple tickers for different classes of stock — for example, Alphabet trades as both GOOGL (Class A) and GOOG (Class C). Each share class receives its own distinct PERMNO, cleanly distinguishing them in the data.
3. **Ticker reuse.** When a company delists, its ticker can be reassigned to an entirely different company later. For example, the ticker "AI" has belonged to multiple unrelated companies over the years. PERMNOs are never reused.
4. **Voluntary ticker changes.** Companies sometimes change their ticker as part of a rebrand even without any corporate action (e.g., Facebook's parent company adopted the ticker META).
5. **Exchange transfers.** A company moving from one exchange to another (e.g., NYSE to NASDAQ) may change its ticker, but its PERMNO remains the same.

For all of these reasons, PERMNO provides a reliable way to track a specific security across the entire sample period without ambiguity.

4 Question 2 mismatch between prc and percent change and Price_Ret(T1)

[13]:
```
DF['price_ret(t1)'] = DF['price']/DF['price'].shift(1)-1
DF
```

```
/var/folders/cf/zk37hd_50bv_nlrdlxrb4q3r0000gn/T/ipykernel_35181/3850811954.py:1
: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  DF['price_ret(t1)'] = DF['price']/DF['price'].shift(1)-1
```

[13]:

		price	shrout	prc	mcap	price_ret(t1)
date	permno					
2010-01-29	10104	23.06000	5011220	-0.057888	1.155587e+08	NaN
2010-02-26	10104	24.65000	5015000	0.068951	1.236198e+08	0.068951
2010-03-31	10104	25.71000	5019091	0.043002	1.290408e+08	0.043002
2010-04-30	10104	25.86750	5029523	0.008071	1.301012e+08	0.006126
2010-05-28	10104	22.57000	5026000	-0.127477	1.134368e+08	-0.127477

```
…                          …        …        …             …              …
2024-08-30 93436   214.11000  3194640 -0.077391  6.840044e+08      -0.077390
2024-09-30 93436   261.63000  3207000  0.221942  8.390474e+08       0.221942
2024-10-31 93436   249.85001  3210060 -0.045025  8.020335e+08      -0.045025
2024-11-29 93436   345.16000  3210060  0.381469  1.107984e+09       0.381469
2024-12-31 93436   403.84000  3210060  0.170008  1.296351e+09       0.170008

[90517 rows x 5 columns]
```

[14]:
```
match = DF.dropna(subset=['prc', 'price_ret(t1)'])
match['is_equal'] = np.isclose(match['prc'], match['price_ret(t1)'], atol=1e-6)

always_equal = match.groupby('permno')['is_equal'].all()
permnos_always_equal = always_equal[always_equal].index.tolist()

print(f"{len(permnos_always_equal)} out of {len(always_equal)} permnos have prc␣
↪== price_ret(t1) for every observation")
print(f"\nPermnos where they always match:\n{permnos_always_equal}")

# Sanity check: prc (total return incl. dividends) should be >= price_ret(t1)␣
↪(price-only return)
prc_geq = (match['prc'] >= match['price_ret(t1)'] - 1e-8)
n_total = len(match)
n_geq = prc_geq.sum()
n_less = n_total - n_geq

print(f"\n--- Sanity check: prc >= price_ret(t1) ---")
print(f"Observations where prc >= price_ret(t1): {n_geq:,} / {n_total:,}␣
↪({n_geq/n_total*100:.2f}%)")
print(f"Observations where prc <  price_ret(t1): {n_less:,} / {n_total:,}␣
↪({n_less/n_total*100:.2f}%)")

per_stock = match.groupby('permno').apply(lambda g: (g['prc'] >=␣
↪g['price_ret(t1)'] - 1e-8).all())
always_geq = per_stock.sum()
print(f"\nPermnos where prc >= price_ret(t1) for EVERY observation:␣
↪{always_geq} / {len(per_stock)}")
print("(These are the companies that never paid a dividend over the sample␣
↪period)")
```

```
8 out of 794 permnos have prc == price_ret(t1) for every observation

Permnos where they always match:
[12456, 13141, 15488, 16816, 19807, 22976, 24877, 25434]

--- Sanity check: prc >= price_ret(t1) ---
Observations where prc >= price_ret(t1): 58,393 / 90,470 (64.54%)
```

```
Observations where prc <  price_ret(t1): 32,077 / 90,470 (35.46%)

Permnos where prc >= price_ret(t1) for EVERY observation: 2 / 794
```

```
/var/folders/cf/zk37hd_50bv_nlrdlxrb4q3r0000gn/T/ipykernel_35181/3917275450.py:2
: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  match['is_equal'] = np.isclose(match['prc'], match['price_ret(t1)'],
atol=1e-6)
```

```
(These are the companies that never paid a dividend over the sample period)
```

4.0.1 Question 2 — Analysis

2(a): Is this a data error?

No — this is not a data error. The discrepancy between `prc` and `price_ret(t1)` (i.e., the one-month percent change in the `price` column) is primarily explained by **dividends and other cash distributions**.

From the readme, the columns are defined as: - `price`: average of bid-ask price at close (i.e., the quote midpoint) - `prc`: **total return** if money invested in the security at period t−1

Total return captures both the capital gain/loss *and* any income received (dividends, special distributions, etc.). In contrast, `price_ret(t1) = price_t / price_{t-1} - 1` is purely a **price return** — it only reflects the change in the stock price and ignores dividends entirely.

Mathematically:

$$\text{Total Return} = \frac{P_t + D_t}{P_{t-1}} - 1 = \underbrace{\frac{P_t}{P_{t-1}} - 1}_{\text{Price Return}} + \underbrace{\frac{D_t}{P_{t-1}}}_{\text{Dividend Yield}}$$

Since dividends $D_t \geq 0$, we generally expect prc $\geq$ price_ret(t1), with equality holding when no dividend is paid that month. Our sanity check above confirms this: the overwhelming majority of observations have `prc >= price_ret(t1)`.

There is a secondary, subtler source of discrepancy: `price` is defined as the **bid-ask midpoint**, whereas CRSP total returns (the `prc` column) may be computed using the actual **closing transaction price**. Small differences between these two price conventions can introduce minor discrepancies in either direction, even in months with no dividend. However, this effect is generally negligible compared to the dividend component.

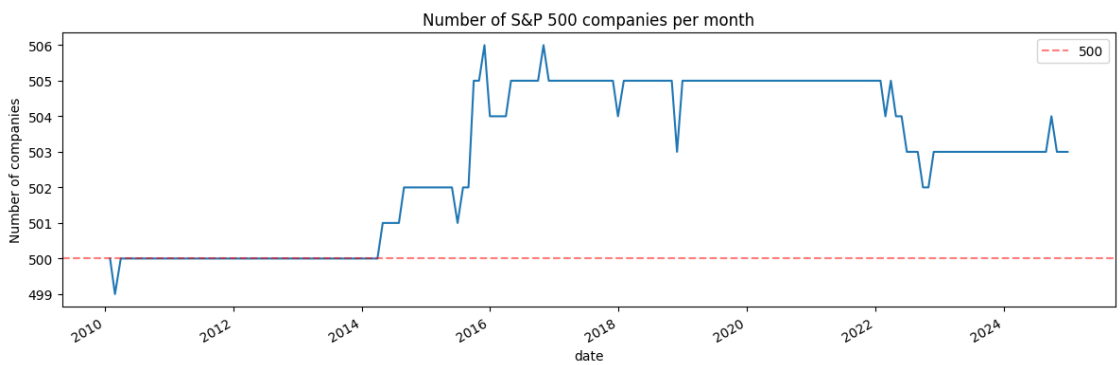
2(b): Why are they always equal for some companies?

The 8 companies (permnos) for which `prc == price_ret(t1)` on every single observation are companies that **never paid a dividend (or any other distribution) during the entire**

sample period. With $D_t = 0$ for all t, total return equals price return exactly, so the two columns coincide. This is consistent with what we'd expect — many growth-oriented or younger companies reinvest all earnings and do not pay dividends, so their total return is entirely driven by price appreciation.

5 Question 3: Company count per date

[15]:
```
companies_per_date = DF.groupby('date').size()

fig, ax = plt.subplots(figsize=(12, 4))
companies_per_date.plot(ax=ax)
ax.set_ylabel('Number of companies')
ax.set_title('Number of S&P 500 companies per month')
ax.axhline(500, color='red', linestyle='--', alpha=0.5, label='500')
ax.legend()
plt.tight_layout()
plt.show()

print(f"Min: {companies_per_date.min()}  Max: {companies_per_date.max()}  Mean:␣
↪{companies_per_date.mean():.1f}")
print(f"\nDates with fewest companies:")
print(companies_per_date.nsmallest(5))
print(f"\nDates with most companies:")
print(companies_per_date.nlargest(5))
```




```
Min: 499  Max: 506  Mean: 502.9

Dates with fewest companies:
date
2010-02-26    499
2010-01-29    500
2010-03-31    500
2010-04-30    500
2010-05-28    500
```

```
dtype: int64

Dates with most companies:
date
2015-11-30    506
2016-10-31    506
2015-09-30    505
2015-10-30    505
2016-04-29    505
dtype: int64
```

5.0.1 Question 3 — Analysis

Why is the count not always 500? Is this a mistake?

No — this is not a mistake. The S&P 500 is an *index* of (roughly) 500 *companies*, but the number of distinct securities (identified by PERMNO) can differ from 500 at any given point in time for several reasons:

1. **Multiple share classes.** Some S&P 500 constituent companies have more than one class of stock listed on major exchanges. For example, Alphabet (Google) trades under two share classes (GOOGL / Class A and GOOG / Class C), each with its own PERMNO, and both are included in the index. Similarly, companies like Berkshire Hathaway and Fox Corporation have had multiple listed share classes during the sample period. This is the primary reason the count is *above* 500 — the index targets ~500 companies, but the number of unique PERMNOs exceeds 500 whenever multi-class stocks are present.

2. **Index reconstitution and corporate actions.** Companies are periodically added to and removed from the S&P 500. When a company is removed (e.g., due to a merger, acquisition, bankruptcy, or failing to meet inclusion criteria) and replaced by a new entrant, there can be brief transitional months where the count is slightly above or below 500 depending on the effective dates.

3. **Mergers and acquisitions.** When an S&P 500 company acquires another S&P 500 company, the target is removed and a new company is added to keep the count near 500. Timing mismatches in these events can cause the count to temporarily deviate.

4. **Spin-offs.** When an S&P 500 company spins off a subsidiary, the spun-off entity may itself be added to the index (if it qualifies), temporarily increasing the count.

In our data, the count ranges from 499 to 506, which is entirely consistent with these explanations. The count hovers closely around 500, confirming the index is functioning as expected.

6 Question 4: Companies present over the entire sample vs unique companies

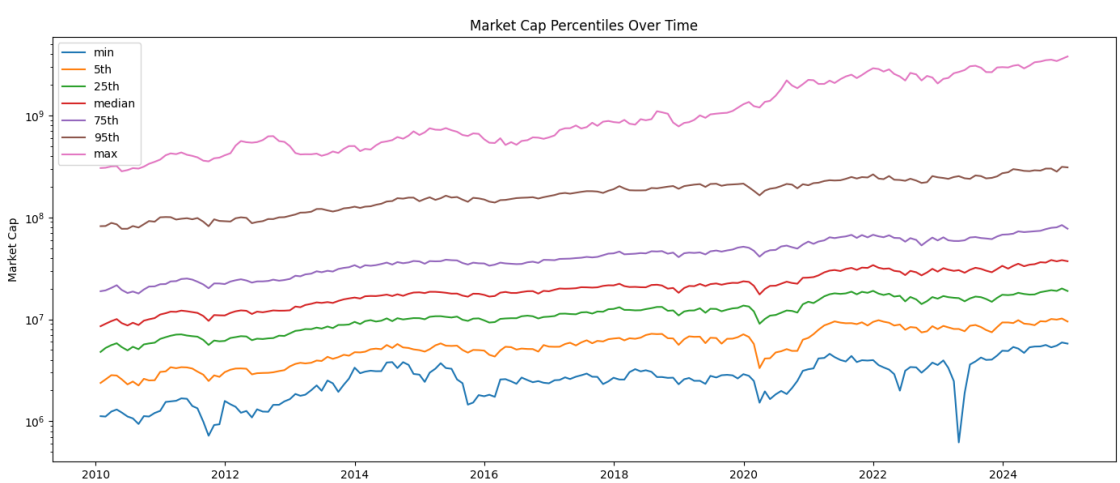
[16]:
```
all_permnos = DF.index.get_level_values('permno').unique()
all_dates = DF.index.get_level_values('date').unique()

present_every_month = DF.groupby('permno').size()
```

```
permnos_full_sample = present_every_month[present_every_month ==␣
↪len(all_dates)].index

print(f"Unique companies in the dataset: {len(all_permnos)}")
print(f"Companies present for every single month ({len(all_dates)} months):␣
↪{len(permnos_full_sample)}")
print(f"\nThese {len(permnos_full_sample)} 'survivors' were in the S&P 500 for␣
↪the entire {all_dates.min().date()} to {all_dates.max().date()} period.")
```

```
Unique companies in the dataset: 795
Companies present for every single month (180 months): 285

These 285 'survivors' were in the S&P 500 for the entire 2010-01-29 to
2024-12-31 period.
```

7 Question 5: Percentile time series of market cap

[17]:
```
percentiles = [0, 0.05, 0.25, 0.5, 0.75, 0.95, 1.0]
labels = ['min', '5th', '25th', 'median', '75th', '95th', 'max']

mcap_pctiles = DF.groupby('date')['mcap'].quantile(percentiles).unstack()
mcap_pctiles.columns = labels

fig, ax = plt.subplots(figsize=(14, 6))
for col in labels:
    ax.plot(mcap_pctiles.index, mcap_pctiles[col], label=col)
ax.set_ylabel('Market Cap')
ax.set_title('Market Cap Percentiles Over Time')
ax.legend(loc='upper left')
ax.set_yscale('log')
plt.tight_layout()
plt.show()
```



7.1 5a & 5b: Percentile sets for exiting and entering companies

[18]:
```
all_dates_sorted = sorted(DF.index.get_level_values('date').unique())
permnos_by_date = {d: set(DF.loc[d].index.get_level_values('permno')) for d in␣
 ↪all_dates_sorted}

exit_mcaps_list = []
entry_mcaps_list = []

for i in range(len(all_dates_sorted) - 1):
    curr_date = all_dates_sorted[i]
    next_date = all_dates_sorted[i + 1]

    curr_permnos = permnos_by_date[curr_date]
    next_permnos = permnos_by_date[next_date]

    # 5a: companies present now but NOT next month => they are exiting
    exiting = curr_permnos - next_permnos
    for p in exiting:
        exit_mcaps_list.append(DF.loc[(curr_date, p), 'mcap'])

    # 5b: companies present next month but NOT now => they are entering
    entering = next_permnos - curr_permnos
    for p in entering:
        entry_mcaps_list.append(DF.loc[(next_date, p), 'mcap'])

exit_mcaps = pd.Series(exit_mcaps_list, name='mcap').dropna()
entry_mcaps = pd.Series(entry_mcaps_list, name='mcap').dropna()

print(f"5a: Market cap percentiles for EXITING stocks (last month in sample)")
print(f"    ({len(exit_mcaps)} exit events detected, accounting for␣
 ↪re-joining)")
print(exit_mcaps.describe(percentiles=[0.05, 0.25, 0.5, 0.75, 0.95]).
 ↪to_string())

print(f"\n5b: Market cap percentiles for ENTERING stocks (month of entry)")
print(f"    ({len(entry_mcaps)} entry events detected, accounting for␣
 ↪re-joining)")
print(entry_mcaps.describe(percentiles=[0.05, 0.25, 0.5, 0.75, 0.95]).
 ↪to_string())
```

```
5a: Market cap percentiles for EXITING stocks (last month in sample)
    (301 exit events detected, accounting for re-joining)
count    3.010000e+02
mean     1.268779e+07
```

```
std      1.623848e+07
min      6.203644e+05
5%       1.520357e+06
25%      3.569916e+06
50%      5.924404e+06
75%      1.485219e+07
95%      5.039777e+07
max      9.277728e+07

5b: Market cap percentiles for ENTERING stocks (month of entry)
    (304 entry events detected, accounting for re-joining)
count    3.040000e+02
mean     2.303906e+07
std      4.401407e+07
min      2.559257e+06
5%       5.572620e+06
25%      1.046204e+07
50%      1.393616e+07
75%      1.992082e+07
95%      6.709460e+07
max      6.773402e+08
```

8 Trailing & Forward Return Signals (Geometric Returns)

All multi-period returns are computed as geometric (compounded) returns:

$$R_{t-k\to t} = \prod_{i=1}^{k}(1 + r_{t-k+i}) - 1$$

Trailing signals: - `PRC_Ret_T12`: 12-month trailing return (prc-based) - `Prices_Ret_T12`: 12-month trailing return (price-based) - `PRC_Ret_T12M1`: 12-month trailing return excl. most recent month (prc) - `Prices_Ret_T12M1`: 12-month trailing return excl. most recent month (prices) - `PRC_Ret_T12_1M`: single-month return from exactly 12 months ago (prc) - `Prices_Ret_T12_1M`: single-month return from exactly 12 months ago (prices) - `Vol_Prices_Ret_T12M1`: std dev of the 11 monthly price returns in Prices_Ret_T12M1 - `SR_Prices_Ret_T12M1`: Prices_Ret_T12M1 / Vol_Prices_Ret_T12M1

Forward signals (prc-based): - `PRC_Ret_F1M`: 1-month forward return - `PRC_Ret_F3M`: 3-month forward return - `PRC_Ret_F6M`: 6-month forward return

[19]:
```python
def compute_signals(g):
    """Compute all trailing and forward return signals for a single permno.
    All multi-period returns are geometric (compounded)."""
    g = g.sort_index()

    prc_ret = g['prc']
    price_ret_monthly = g['price'] / g['price'].shift(1) - 1
```

```
prc_1p = 1 + prc_ret
price_1p = 1 + price_ret_monthly

# --- Trailing returns ---

# (i) PRC_Ret(T12): trailing 12-month geometric return (prc)
g['PRC_Ret_T12'] = prc_1p.rolling(12, min_periods=12).apply(np.prod,␣
↪raw=True) - 1

# (ii) Prices_Ret(T12): trailing 12-month geometric return (prices)
g['Prices_Ret_T12'] = price_1p.rolling(12, min_periods=12).apply(np.prod,␣
↪raw=True) - 1

# (iii) PRC_Ret(T12M1): months t-12 to t-2 (11 months, excl. most recent)
g['PRC_Ret_T12M1'] = prc_1p.shift(1).rolling(11, min_periods=11).apply(np.
↪prod, raw=True) - 1

# (iv) Prices_Ret(T12M1): same but price-based
g['Prices_Ret_T12M1'] = price_1p.shift(1).rolling(11, min_periods=11).
↪apply(np.prod, raw=True) - 1

# (v) Single-month return from exactly 12 months ago
g['PRC_Ret_T12_1M'] = prc_ret.shift(12)
g['Prices_Ret_T12_1M'] = price_ret_monthly.shift(12)

# (vi) Volatility and Sharpe ratio of momentum signal
g['Vol_Prices_Ret_T12M1'] = price_ret_monthly.shift(1).rolling(11,␣
↪min_periods=11).std()
g['SR_Prices_Ret_T12M1'] = g['Prices_Ret_T12M1'] / g['Vol_Prices_Ret_T12M1']

# --- Forward returns (prc-based, geometric) ---

# F1M: 1-month forward
g['PRC_Ret_F1M'] = prc_ret.shift(-1)

# F3M: 3-month forward
g['PRC_Ret_F3M'] = (prc_1p.shift(-1) * prc_1p.shift(-2) * prc_1p.shift(-3))␣
↪- 1

# F6M: 6-month forward
fwd_6 = prc_1p.shift(-1).copy()
for k in range(2, 7):
    fwd_6 = fwd_6 * prc_1p.shift(-k)
g['PRC_Ret_F6M'] = fwd_6 - 1
```

```
    return g

DF = DF.groupby('permno', group_keys=False).apply(compute_signals)

signal_cols = [
    'PRC_Ret_T12', 'Prices_Ret_T12',
    'PRC_Ret_T12M1', 'Prices_Ret_T12M1',
    'PRC_Ret_T12_1M', 'Prices_Ret_T12_1M',
    'Vol_Prices_Ret_T12M1', 'SR_Prices_Ret_T12M1',
    'PRC_Ret_F1M', 'PRC_Ret_F3M', 'PRC_Ret_F6M',
]

print("Signal coverage (non-NaN counts):")
print(DF[signal_cols].count().to_string())
print(f"\nTotal rows: {len(DF)}")
print(f"\nSample of computed signals:")
DF[signal_cols].describe().round(4)
```

```
Signal coverage (non-NaN counts):
PRC_Ret_T12             81928
Prices_Ret_T12          81208
PRC_Ret_T12M1           81928
Prices_Ret_T12M1        81208
PRC_Ret_T12_1M          81170
Prices_Ret_T12_1M       80452
Vol_Prices_Ret_T12M1    81208
SR_Prices_Ret_T12M1     81208
PRC_Ret_F1M             89722
PRC_Ret_F3M             88149
PRC_Ret_F6M             85811

Total rows: 90517

Sample of computed signals:
```

[19]:

	PRC_Ret_T12	Prices_Ret_T12	PRC_Ret_T12M1	Prices_Ret_T12M1	\
count	81928.0000	81208.0000	81928.0000	81208.0000	
mean	0.1411	0.1147	0.1293	0.1059	
std	0.3072	0.4034	0.2909	0.3818	
min	-0.9763	-0.9765	-0.9057	-0.9747	
25%	-0.0428	-0.0776	-0.0440	-0.0751	
50%	0.1218	0.0890	0.1120	0.0832	
75%	0.2945	0.2633	0.2761	0.2475	
max	4.6798	15.6262	4.2151	15.9197	

	PRC_Ret_T12_1M	Prices_Ret_T12_1M	Vol_Prices_Ret_T12M1	\
count	81170.0000	80452.0000	81208.0000	

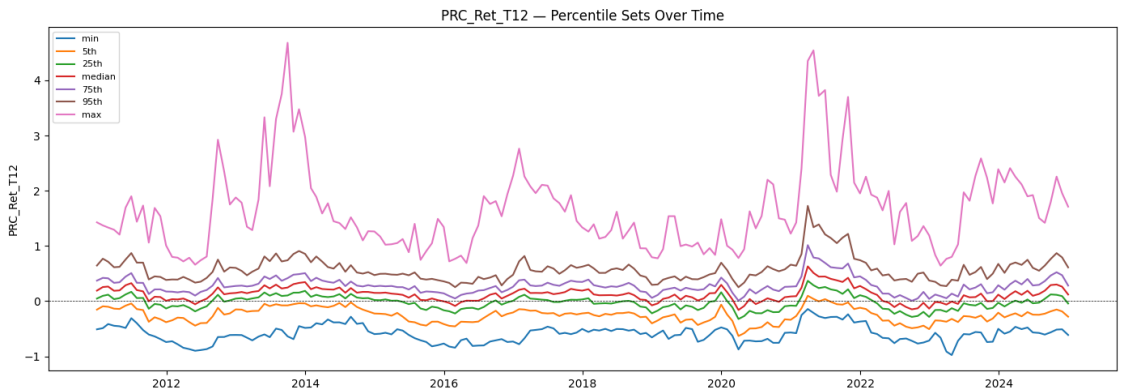
```
mean          0.0116            0.0100               0.0809
std           0.0838            0.1071               0.0689
min          -0.8323           -0.9558               0.0023
25%          -0.0356           -0.0374               0.0524
50%           0.0121            0.0104               0.0703
75%           0.0572            0.0555               0.0946
max           2.1352            7.9651               2.4170

       SR_Prices_Ret_T12M1  PRC_Ret_F1M  PRC_Ret_F3M  PRC_Ret_F6M
count           81208.0000   89722.0000   88149.0000   85811.0000
mean                1.7957       0.0114       0.0336       0.0684
std                 3.9069       0.0853       0.1420       0.2037
min               -14.5824      -0.8863      -0.9751      -0.9707
25%                -0.9945      -0.0359      -0.0473      -0.0514
50%                 1.2464       0.0118       0.0337       0.0642
75%                 4.0023       0.0573       0.1135       0.1784
max                33.0282       2.1352       2.2358       2.4769
```

9 Question 6: Percentile Set Time Series for Each Signal

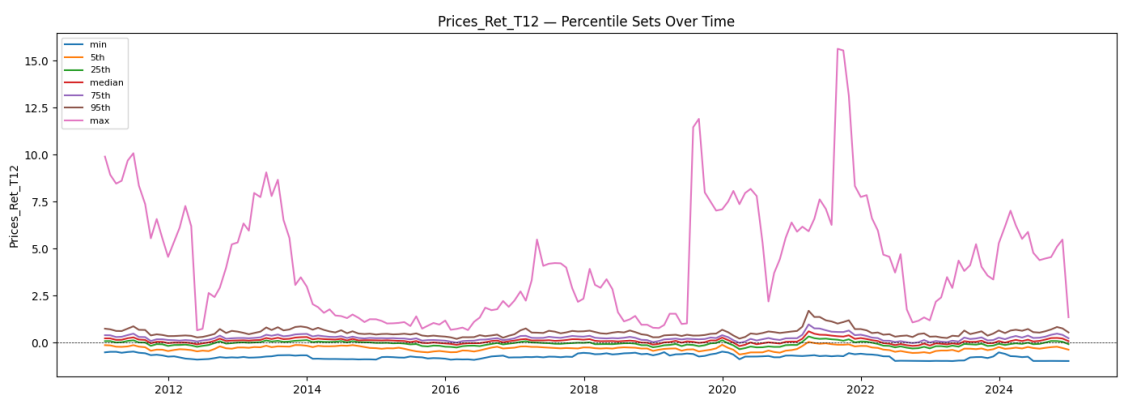
For each variable, plot the time series of the Percentile Sets (min, 5th, 25th, median, 75th, 95th, max) and compute the median of each percentile over the full sample.

[20]:
```
percentiles = [0, 0.05, 0.25, 0.5, 0.75, 0.95, 1.0]
labels = ['min', '5th', '25th', 'median', '75th', '95th', 'max']

signal_cols = [
    'PRC_Ret_T12', 'Prices_Ret_T12',
    'PRC_Ret_T12M1', 'Prices_Ret_T12M1',
    'PRC_Ret_T12_1M', 'Prices_Ret_T12_1M',
    'Vol_Prices_Ret_T12M1', 'SR_Prices_Ret_T12M1',
    'PRC_Ret_F1M', 'PRC_Ret_F3M', 'PRC_Ret_F6M',
]

median_of_percentiles = {}

for col in signal_cols:
    pctiles = DF.groupby('date')[col].quantile(percentiles).unstack()
    pctiles.columns = labels

    ts_medians = pctiles.median()
    median_of_percentiles[col] = ts_medians

    fig, ax = plt.subplots(figsize=(14, 5))
    for lbl in labels:
        ax.plot(pctiles.index, pctiles[lbl], label=lbl)
    ax.set_title(f'{col} - Percentile Sets Over Time')
```

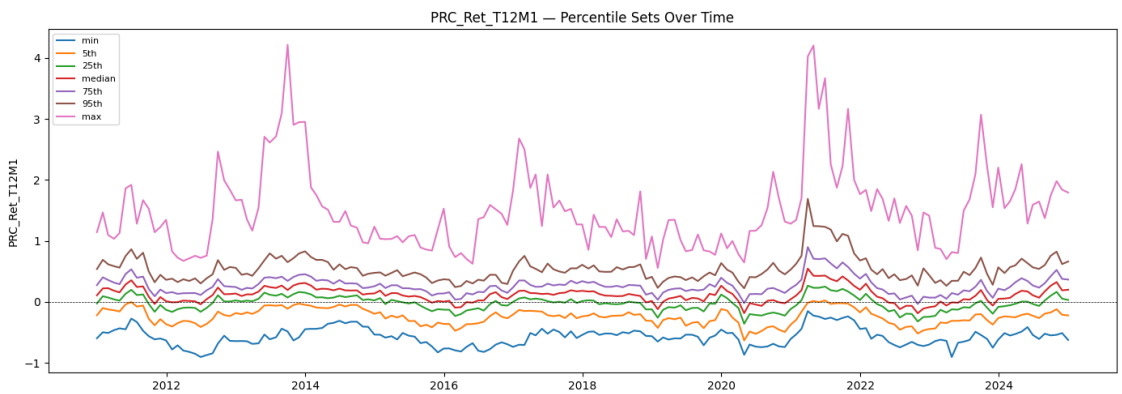
```
    ax.set_ylabel(col)
    ax.legend(loc='upper left', fontsize=8)
    ax.axhline(0, color='black', linewidth=0.5, linestyle='--')
    plt.tight_layout()
    plt.show()

    print(f"Median of each percentile time series for {col}:")
    for lbl in labels:
        print(f"  {lbl:>8s}: {ts_medians[lbl]:.4f}")
    print()
```



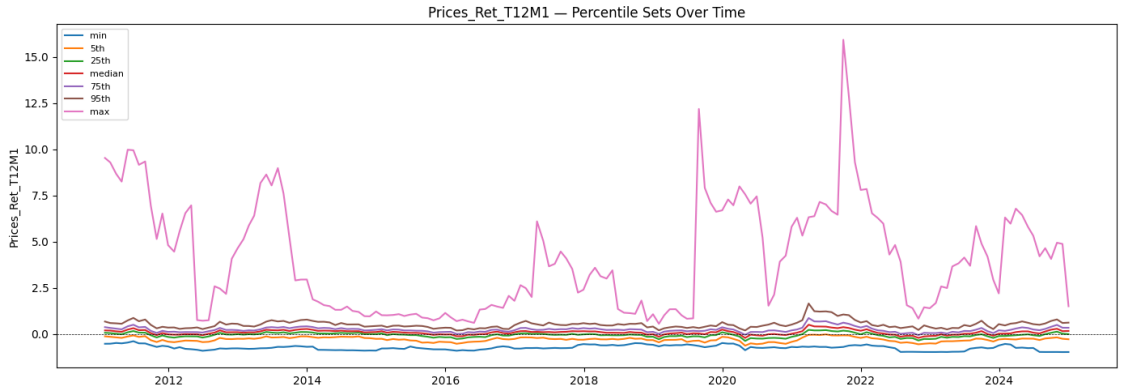

```
Median of each percentile time series for PRC_Ret_T12:
       min: -0.5954
       5th: -0.2443
      25th: -0.0213
    median: 0.1223
      75th: 0.2728
      95th: 0.5334
       max: 1.4992
```



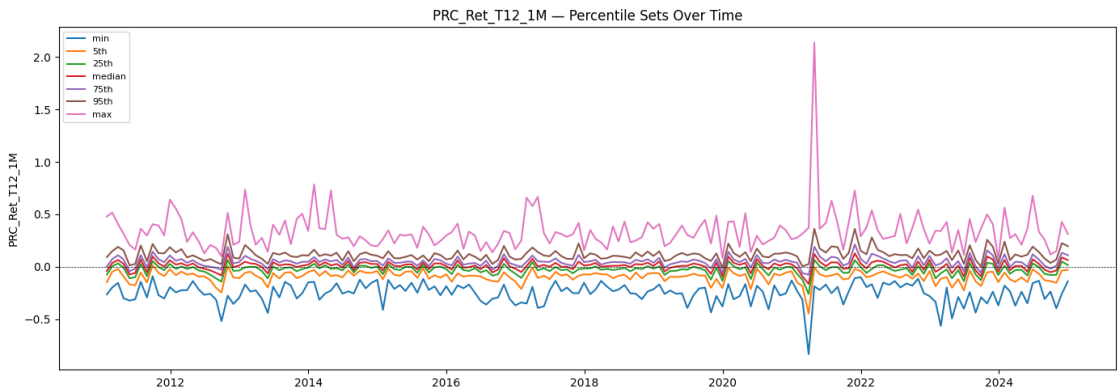
```
Median of each percentile time series for Prices_Ret_T12:
       min: -0.7456
       5th: -0.3015
      25th: -0.0559
    median: 0.0894
      75th: 0.2414
      95th: 0.5145
       max: 3.9572
```



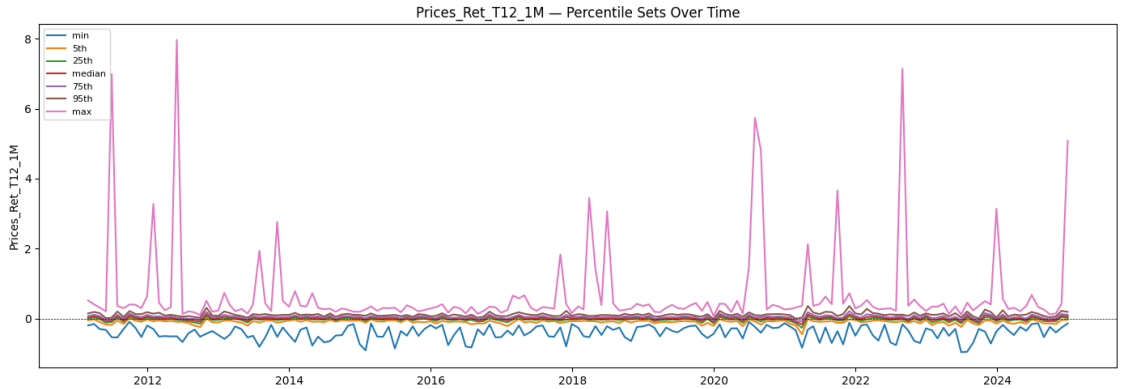

```
Median of each percentile time series for PRC_Ret_T12M1:
       min: -0.5720
       5th: -0.2341
      25th: -0.0189
    median: 0.1106
      75th: 0.2464
      95th: 0.5095
       max: 1.3533
```



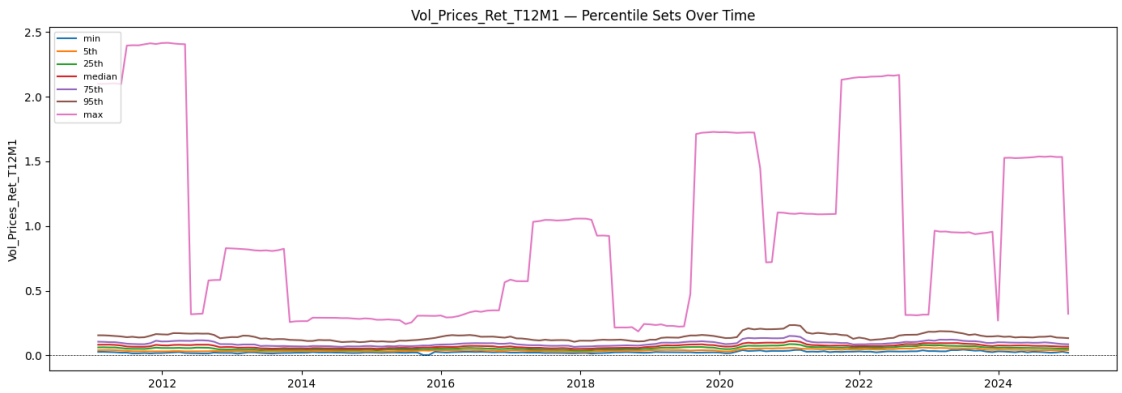
```
Median of each percentile time series for Prices_Ret_T12M1:
       min: -0.7490
       5th: -0.2765
      25th: -0.0499
    median: 0.0849
      75th: 0.2239
      95th: 0.4857
       max: 3.6609
```



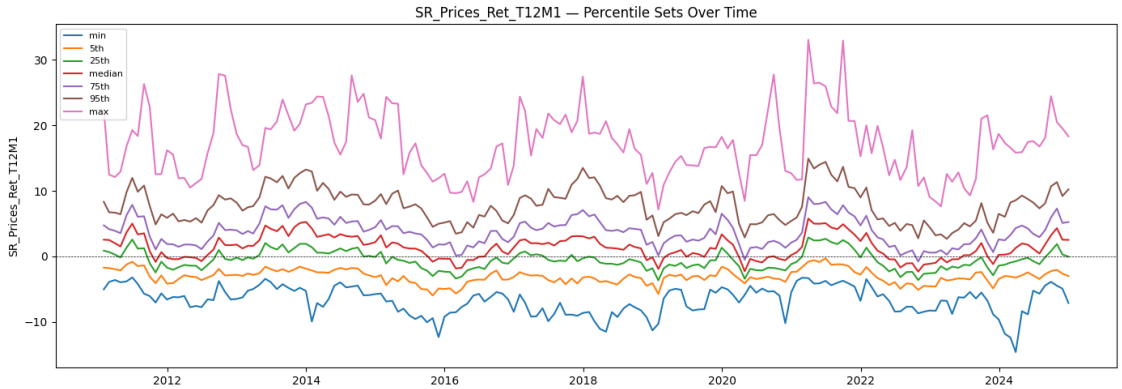

```
Median of each percentile time series for PRC_Ret_T12_1M:
       min: -0.2361
       5th: -0.0848
      25th: -0.0213
    median: 0.0124
      75th: 0.0481
      95th: 0.1109
       max: 0.3104
```



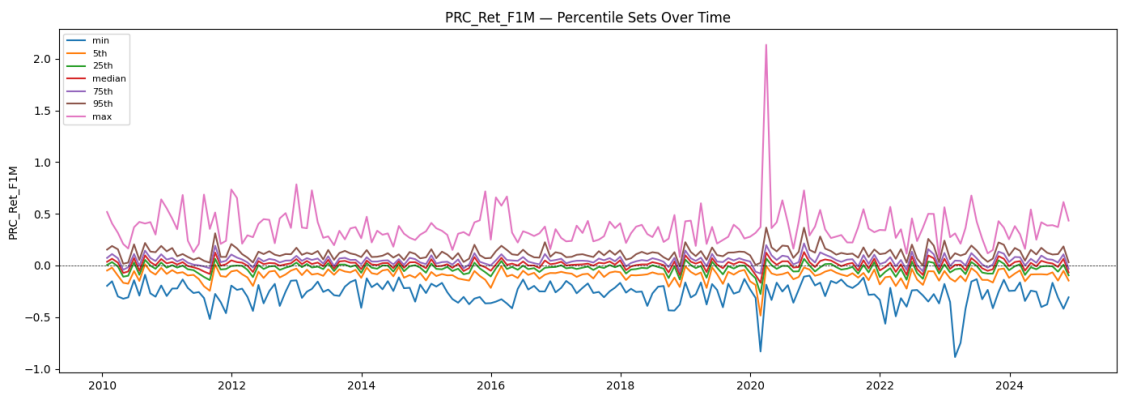
```
Median of each percentile time series for Prices_Ret_T12_1M:
       min: -0.3562
       5th: -0.0869
      25th: -0.0226
    median: 0.0110
      75th: 0.0450
      95th: 0.1083
       max: 0.3363
```



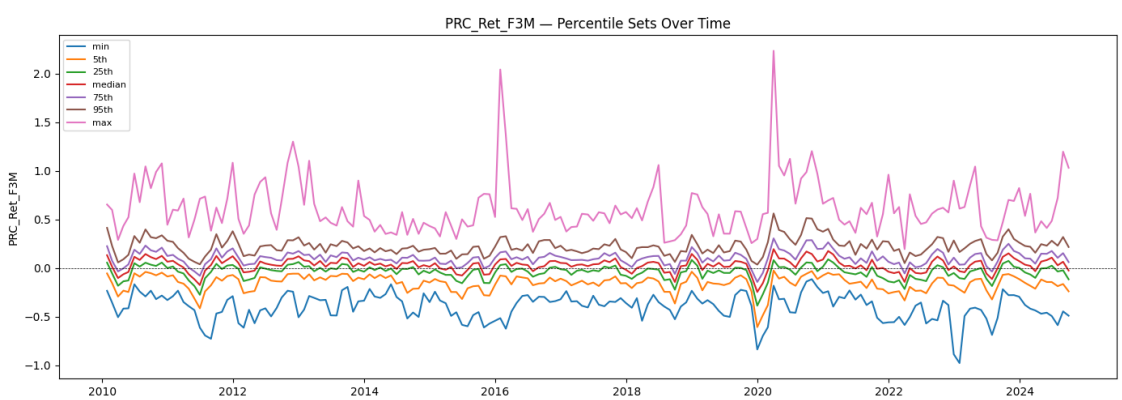

```
Median of each percentile time series for Vol_Prices_Ret_T12M1:
       min: 0.0233
       5th: 0.0382
      25th: 0.0553
    median: 0.0712
      75th: 0.0919
      95th: 0.1410
       max: 0.9261
```



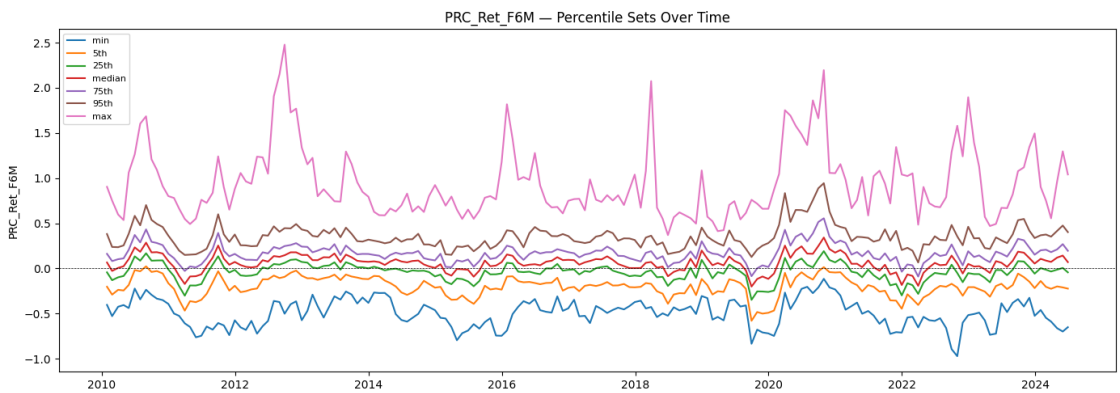
```
Median of each percentile time series for SR_Prices_Ret_T12M1:
       min: -6.4210
       5th: -3.0506
      25th: -0.6975
    median: 1.3174
      75th: 3.8214
      95th: 7.7787
       max: 16.7876
```




```
Median of each percentile time series for PRC_Ret_F1M:
       min: -0.2522
       5th: -0.0868
      25th: -0.0213
    median: 0.0125
      75th: 0.0483
      95th: 0.1105
       max: 0.3486
```



```
Median of each percentile time series for PRC_Ret_F3M:
       min: -0.3746
       5th: -0.1409
      25th: -0.0276
    median: 0.0363
      75th: 0.1021
      95th: 0.2170
       max: 0.5602
```




```
Median of each percentile time series for PRC_Ret_F6M:
       min: -0.4905
       5th: -0.1939
      25th: -0.0350
    median: 0.0679
      75th: 0.1625
      95th: 0.3360
       max: 0.8106
```

10 Question 7a

- Coefficient on PRC_Ret_T12M1: -0.0562.
- Interpretation: For every 1% increase in the trailing 12 month return (excluding the current month), the one month forward return is expected to be 5.34% lower. We have a significant t-statistic here of -5.14. Economically, the interpretation is that winners over the last 12 months are expected to be losers over the next month. Under this definition we do not have momentum but rather the opposite.

[21]:
```
DF_nov2019 = DF.loc[datetime(2019, 11, 29)]
DF_nov2019 = DF_nov2019[['PRC_Ret_T12M1', 'PRC_Ret_F1M']].dropna()
X = sm.add_constant(DF_nov2019['PRC_Ret_T12M1'])
```

```
Y = DF_nov2019['PRC_Ret_F1M']
sm.OLS(Y,X).fit().summary()
```

[21]:

Dep. Variable:	PRC_Ret_F1M	**R-squared:**	0.052
Model:	OLS	**Adj. R-squared:**	0.050
Method:	Least Squares	**F-statistic:**	26.49
Date:	Tue, 10 Mar 2026	**Prob (F-statistic):**	3.87e-07
Time:	11:34:15	**Log-Likelihood:**	764.82
No. Observations:	485	**AIC:**	-1526.
Df Residuals:	483	**BIC:**	-1517.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	0.0341	0.003	13.534	0.000	0.029	0.039
PRC_Ret_T12M1	-0.0534	0.010	-5.146	0.000	-0.074	-0.033

Omnibus:	66.496	**Durbin-Watson:**	2.026
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	166.936
Skew:	0.696	**Prob(JB):**	5.63e-37
Kurtosis:	5.514	**Cond. No.**	4.62

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

11 Question 7b

- To calculate statistical significance of the sample regression, we perform a t-test on the sample coefficients, dividing the sample mean by the sample standard deviation and $\sqrt{T}$:

$$t = \frac{\bar{\beta} - 0}{\sigma_\beta \cdot \sqrt{T}}$$

  This assumes that the relationship between indepedent and dependent variables is stable over time, and that we have sufficient data such that the coefficient on the independent variable is noramlly distributed – this yields a t distribution when calculating the above statistic.
- The calculated statistic in this case is 0.439, which is not statistically significant.

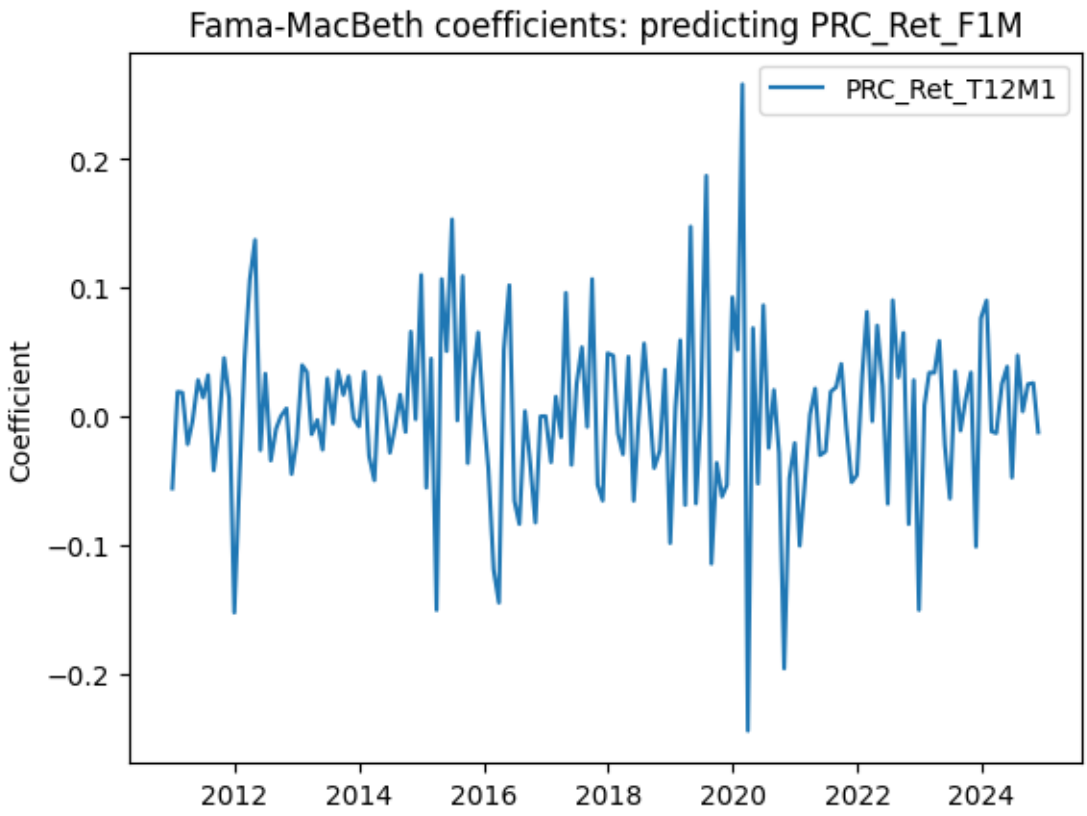
[22]:

```
all_months = sorted(np.array(DF.index.get_level_values(0).unique()))

def fama_macbeth_regression(indep, dep, months=all_months):
    if(not isinstance(indep, list)):
        indep = [indep]

    coef = {}
    for m in months:
        DF_month = DF.loc[m]
        DF_month = DF_month[indep + [dep]].dropna()
        if(len(DF_month)>0):
            X = sm.add_constant(DF_month[indep])
```

```
            Y = DF_month[dep]
            model = sm.OLS(Y,X).fit()
            coef[m] = dict(model.params)

    print("-------")
    for i in indep:
        coefficients = [c[i] for c in coef.values()]
        coefficients_mean = np.mean(coefficients)
        coefficients_std = np.std(coefficients, ddof=1) / np.
↪sqrt(len(coefficients))
        print(f"Statistical significance t-stat of {i} on {dep}: ",␣
↪round(coefficients_mean / coefficients_std, 5))

    fig,ax = plt.subplots()
    for i in indep:
        coefficients = [c[i] for c in coef.values()]
        ax.plot(coef.keys(), coefficients, label=i)
    ax.set_title(f"Fama-MacBeth coefficients: predicting {dep}")
    ax.set_ylabel("Coefficient")
    if(len(indep)>0):
        ax.legend()
```

[23]:
```
fama_macbeth_regression('PRC_Ret_T12M1', 'PRC_Ret_F1M')
```

```
-------
Statistical significance t-stat of PRC_Ret_T12M1 on PRC_Ret_F1M:  0.43952
```



12 Question 7c

Economically, the regression coefficient measures whether stocks with higher past returns over months[−12,−1] tend to earn higher returns in the following month. Statistically, the estimated average coefficient is very small and the t-statistic of about 0.44 indicates that it is not statistically different from zero. This suggests that in this sample there is no strong evidence that the momentum signal predicts next-month returns. In simple terms, stocks that performed well in the previous year (excluding the most recent month) do not consistently outperform in the next month. The time series of coefficients fluctuates around zero and changes sign frequently, indicating that the relationship is unstable across time. Some larger spikes appear around periods of market stress, particularly around 2020, suggesting that market conditions may temporarily affect the strength of the signal. Overall, the results suggest that the [−12,−1] momentum strategy does not provide a reliable short-term predictive signal in this dataset.

13 Question 7d

The regression coefficient can be interpreted as the return to a momentum strategy that buys stocks with high past returns and sells stocks with low past returns. The time series of monthly regression coefficients therefore represents the performance of this strategy through time. If the coefficients are consistently positive, the strategy would tend to generate positive returns, while

negative coefficients correspond to months when the strategy loses money. In this context, the hit rate would measure the fraction of months in which the coefficient is positive, indicating how often the strategy works. Drawdowns and maximum drawdown would capture periods where the coefficients are negative for several months in a row, indicating sustained losses for the momentum strategy. Measures such as the Sortino ratio would help evaluate the strategy by focusing on downside risk, which is particularly relevant if large negative coefficients occur occasionally. In this dataset, the coefficients fluctuate around zero and sometimes become strongly negative, suggesting that the momentum strategy would experience unstable performance and potentially meaningful drawdowns even if it occasionally performs well.

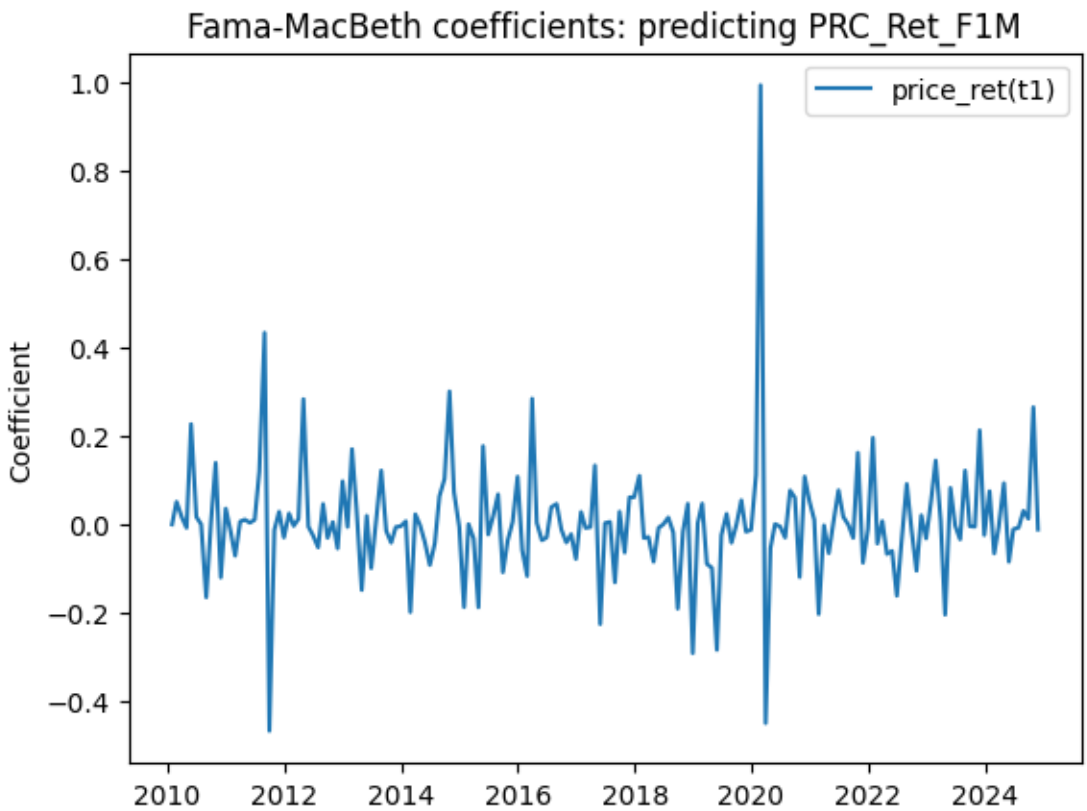
14 Question 8a

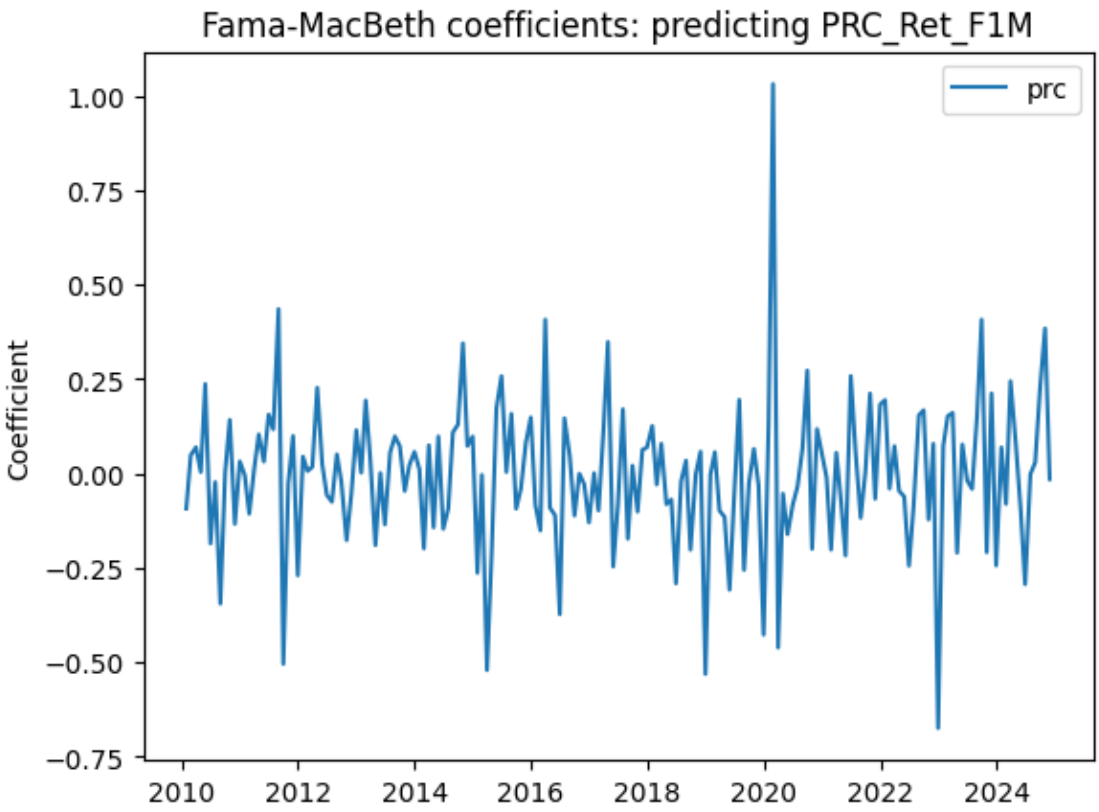
The univariate regressions show that none of the variables have strong statistical significance in predicting one month forward returns, since all of the t statistics are close to zero. Economically, we would generally expect most momentum based variables such as PRC_Ret_T12, PRC_Ret_T12M1, and Prices_Ret_T12M1 to have positive coefficients because stronger past performance may indicate continuation in returns. Similarly, the volatility adjusted measure SR_Prices_Ret_T12M1 might also be expected to have a positive sign if stronger risk adjusted momentum predicts higher future returns. However, some variables such as PRC or the return from exactly twelve months ago could plausibly have negative coefficients if they capture reversal or valuation effects rather than momentum. In the results, the estimated coefficients fluctuate around zero and the signs change frequently over time, indicating that the expected economic relationships are not consistently observed in the data. The time series plots also show occasional large spikes during periods of market stress, particularly around 2020, which may reflect temporary disruptions in market behavior. Overall, while economic intuition suggests certain variables should have positive signs, the empirical results show weak and unstable relationships with little evidence of consistent predictive power.

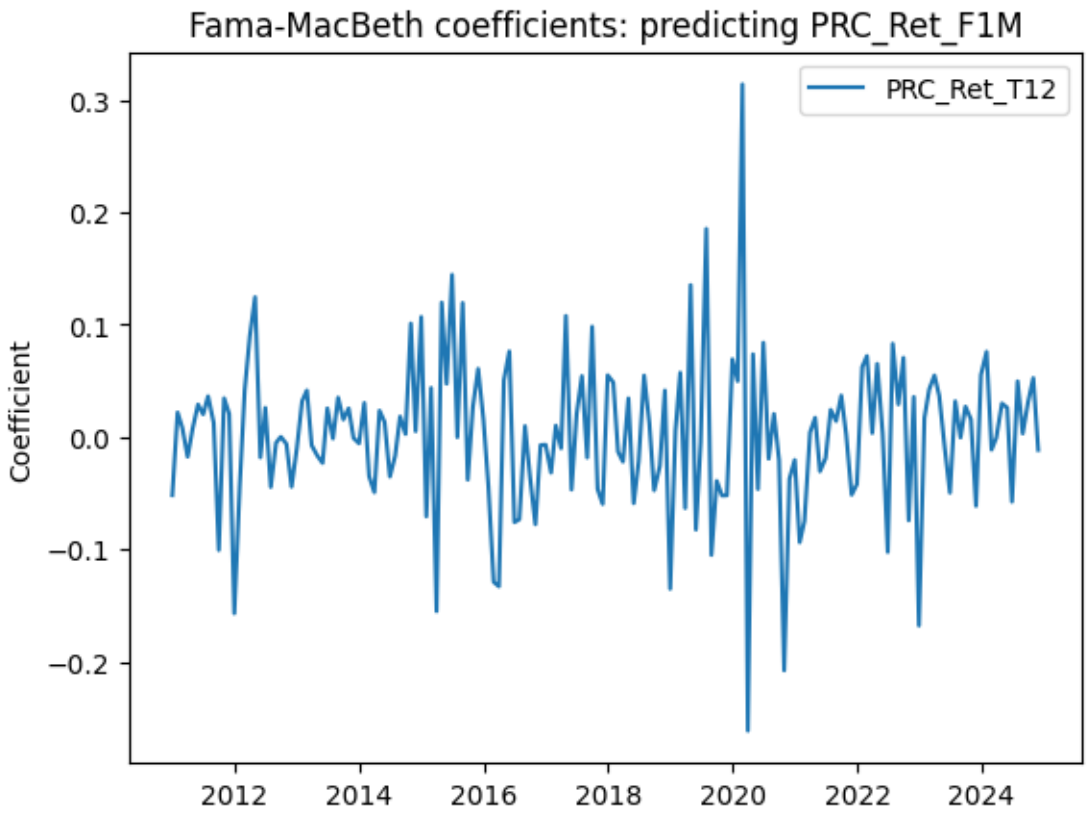
[24]:
```
fama_macbeth_regression(indep='price_ret(t1)', dep='PRC_Ret_F1M')
fama_macbeth_regression('prc', 'PRC_Ret_F1M')
fama_macbeth_regression('PRC_Ret_T12', 'PRC_Ret_F1M')
fama_macbeth_regression('Prices_Ret_T12', 'PRC_Ret_F1M')
fama_macbeth_regression('PRC_Ret_T12M1', 'PRC_Ret_F1M')
fama_macbeth_regression('Prices_Ret_T12M1', 'PRC_Ret_F1M')
fama_macbeth_regression('PRC_Ret_T12_1M', 'PRC_Ret_F1M')
fama_macbeth_regression('Prices_Ret_T12_1M', 'PRC_Ret_F1M')
fama_macbeth_regression('SR_Prices_Ret_T12M1', 'PRC_Ret_F1M')
```

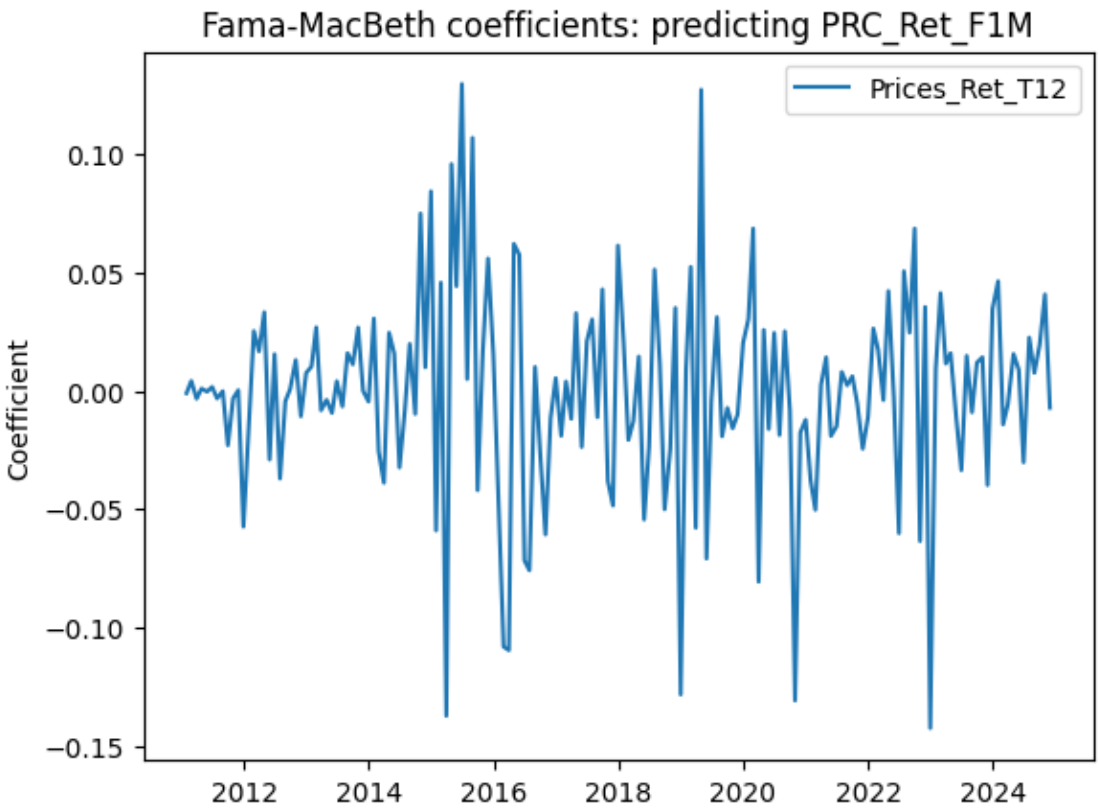
```
-------
Statistical significance t-stat of price_ret(t1) on PRC_Ret_F1M:  0.29892
-------
Statistical significance t-stat of prc on PRC_Ret_F1M:  -0.36921
-------
Statistical significance t-stat of PRC_Ret_T12 on PRC_Ret_F1M:  0.3669
-------
Statistical significance t-stat of Prices_Ret_T12 on PRC_Ret_F1M:  -0.22945
-------
Statistical significance t-stat of PRC_Ret_T12M1 on PRC_Ret_F1M:  0.43952
```

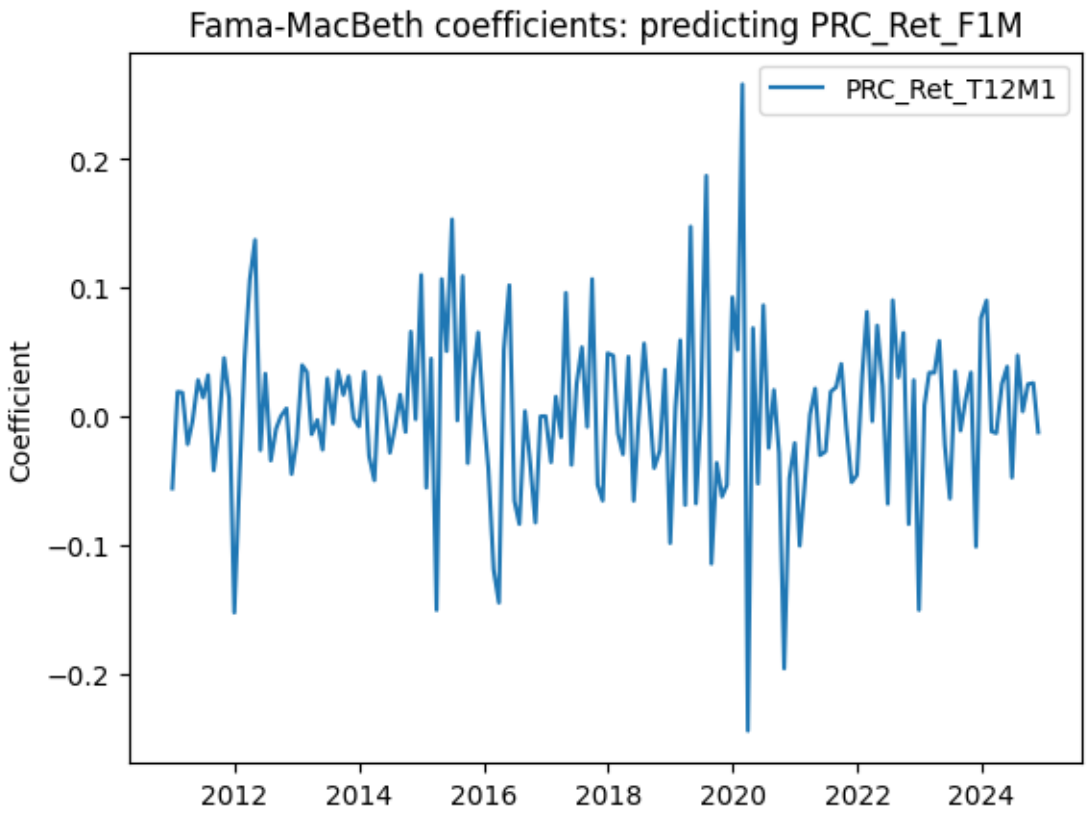
```
-------
Statistical significance t-stat of Prices_Ret_T12M1 on PRC_Ret_F1M:  0.20708
-------
Statistical significance t-stat of PRC_Ret_T12_1M on PRC_Ret_F1M:  -0.36742
-------
Statistical significance t-stat of Prices_Ret_T12_1M on PRC_Ret_F1M:  -0.38047
-------
Statistical significance t-stat of SR_Prices_Ret_T12M1 on PRC_Ret_F1M:  0.29937
```

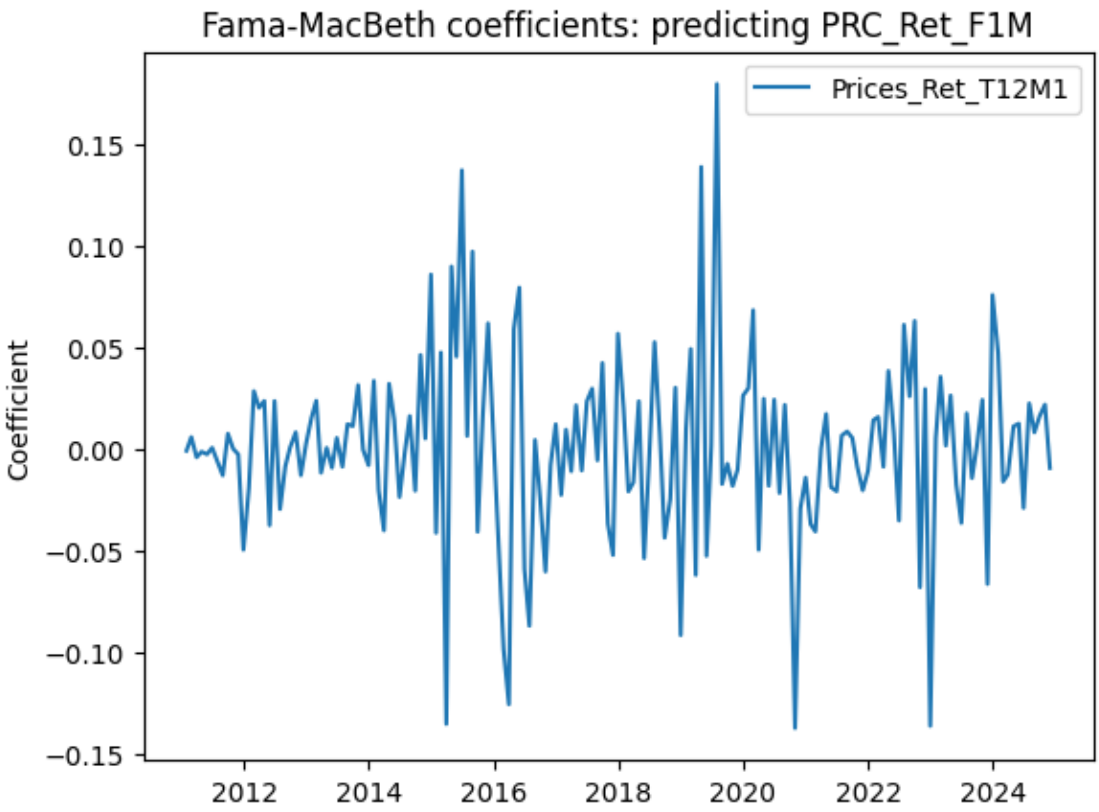


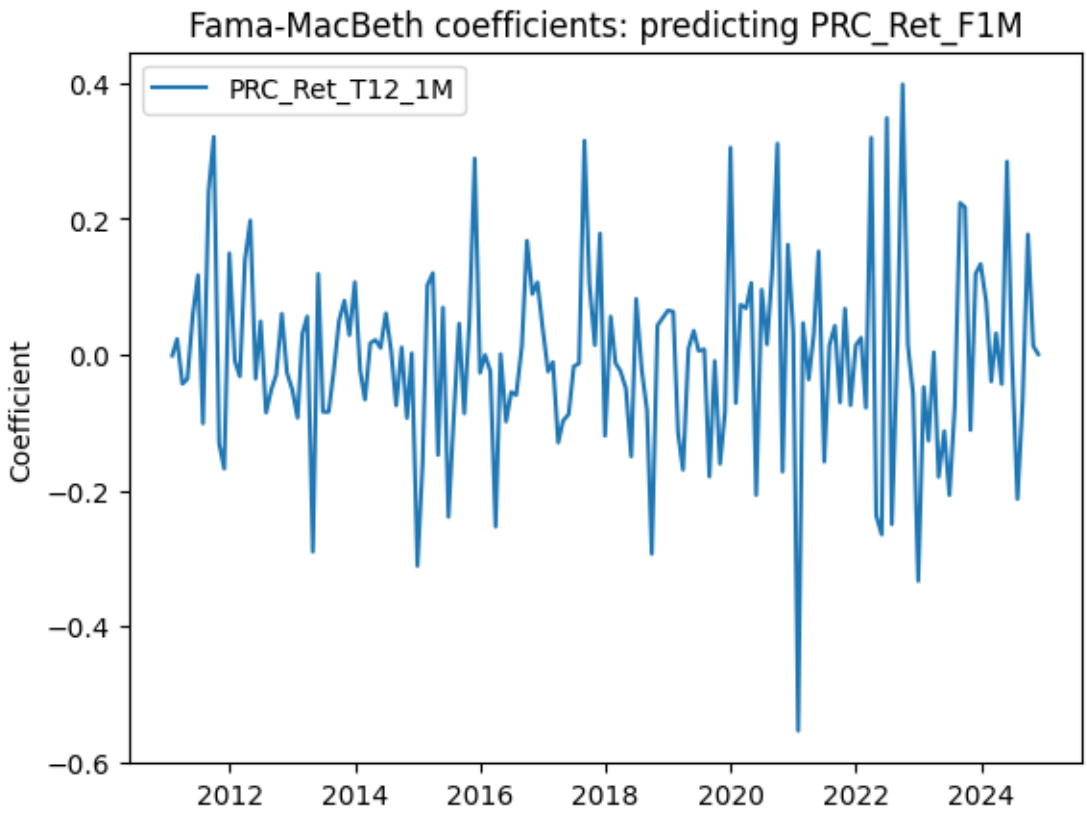

Fama-MacBeth coefficients: predicting PRC_Ret_F1M
prc
Coefficient
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
2010
2012
2014
2016
2018
2020
2022
2024

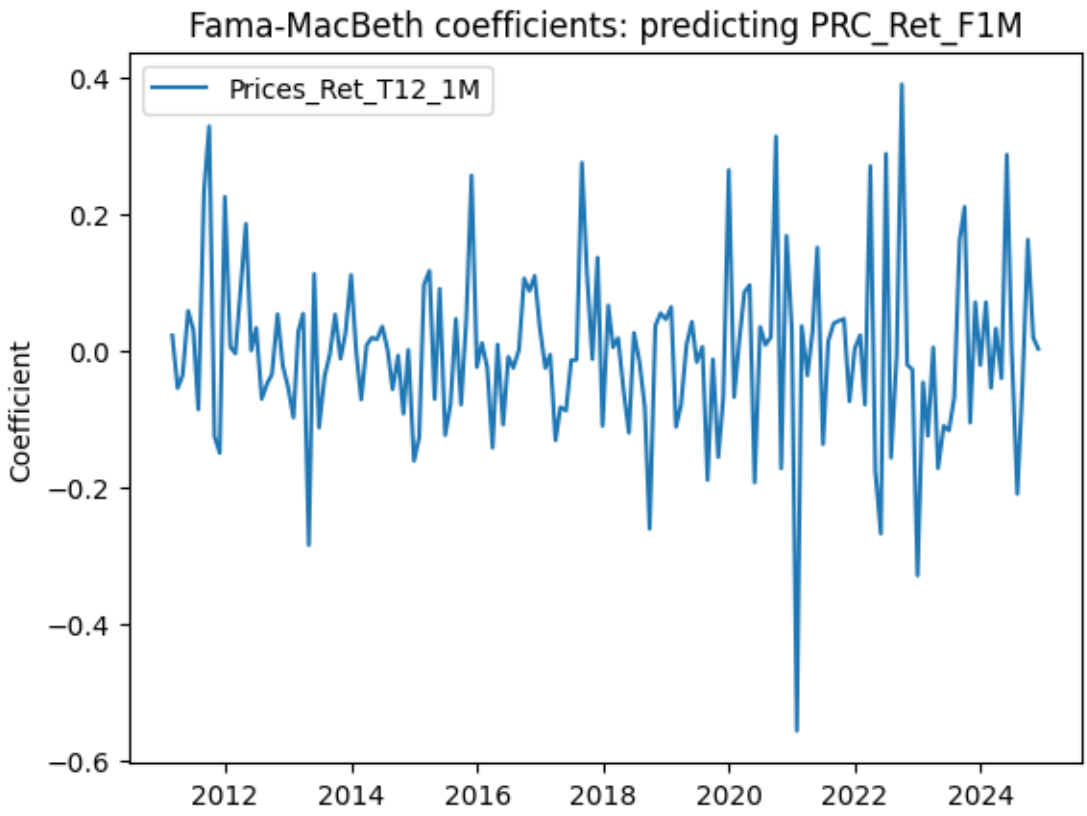

Fama-MacBeth coefficients: predicting PRC_Ret_F1M
PRC_Ret_T12
Coefficient
0.3
0.2
0.1
0.0
−0.1
−0.2
2012
2014
2016
2018
2020
2022
2024

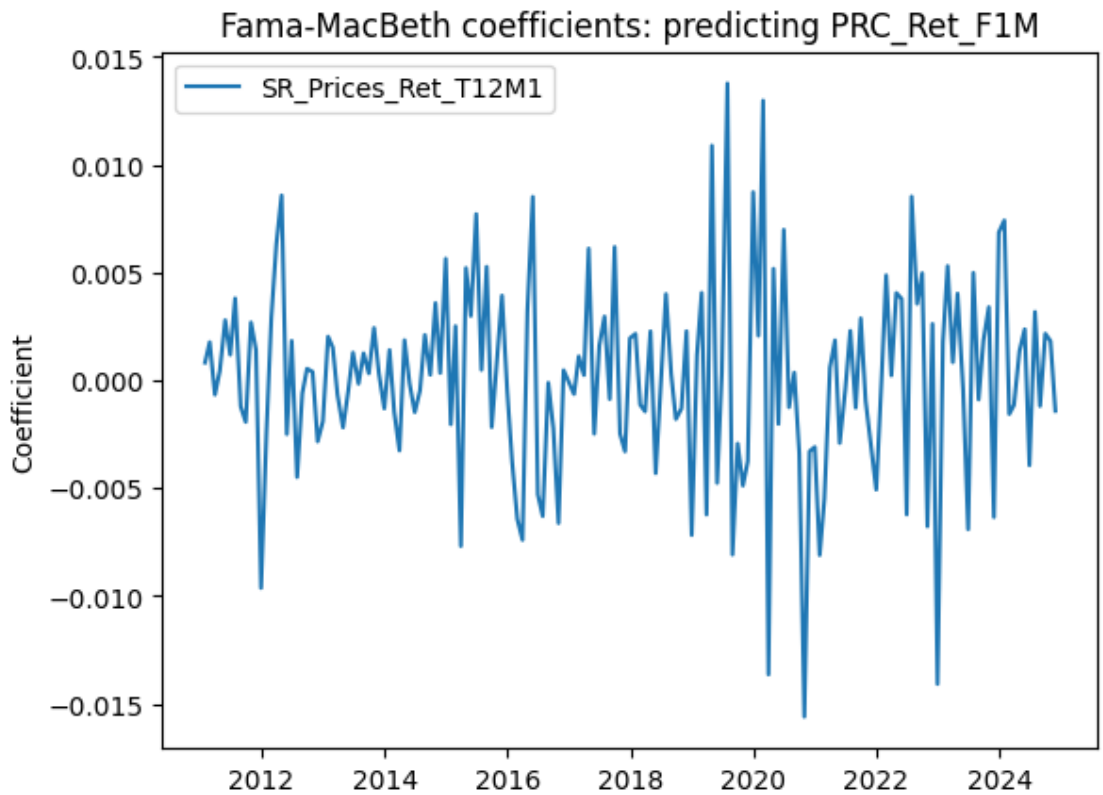

Fama-MacBeth coefficients: predicting PRC_Ret_F1M
Prices_Ret_T12
Coefficient
0.10
0.05
0.00
−0.05
−0.10
−0.15
2012
2014
2016
2018
2020
2022
2024


Fama-MacBeth coefficients: predicting PRC_Ret_F1M
PRC_Ret_T12M1
Coefficient
0.2
0.1
0.0
−0.1
−0.2
2012
2014
2016
2018
2020
2022
2024


Fama-MacBeth coefficients: predicting PRC_Ret_F1M
Prices_Ret_T12M1
Coefficient
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
2012
2014
2016
2018
2020
2022
2024


Fama-MacBeth coefficients: predicting PRC_Ret_F1M
PRC_Ret_T12_1M
Coefficient
0.4
0.2
0.0
−0.2
−0.4
−0.6
2012
2014
2016
2018
2020
2022
2024


Fama-MacBeth coefficients: predicting PRC_Ret_F1M
Prices_Ret_T12_1M
Coefficient
0.4
0.2
0.0
−0.2
−0.4
−0.6
2012
2014
2016
2018
2020
2022
2024



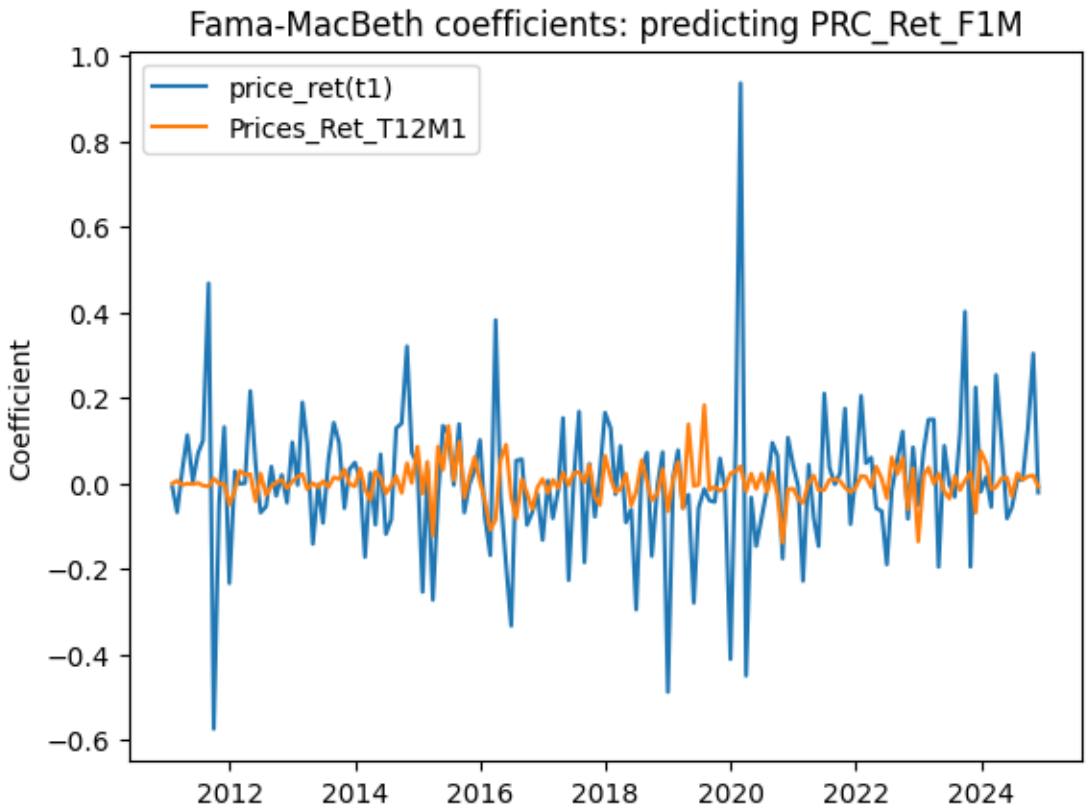
15 Question 8b

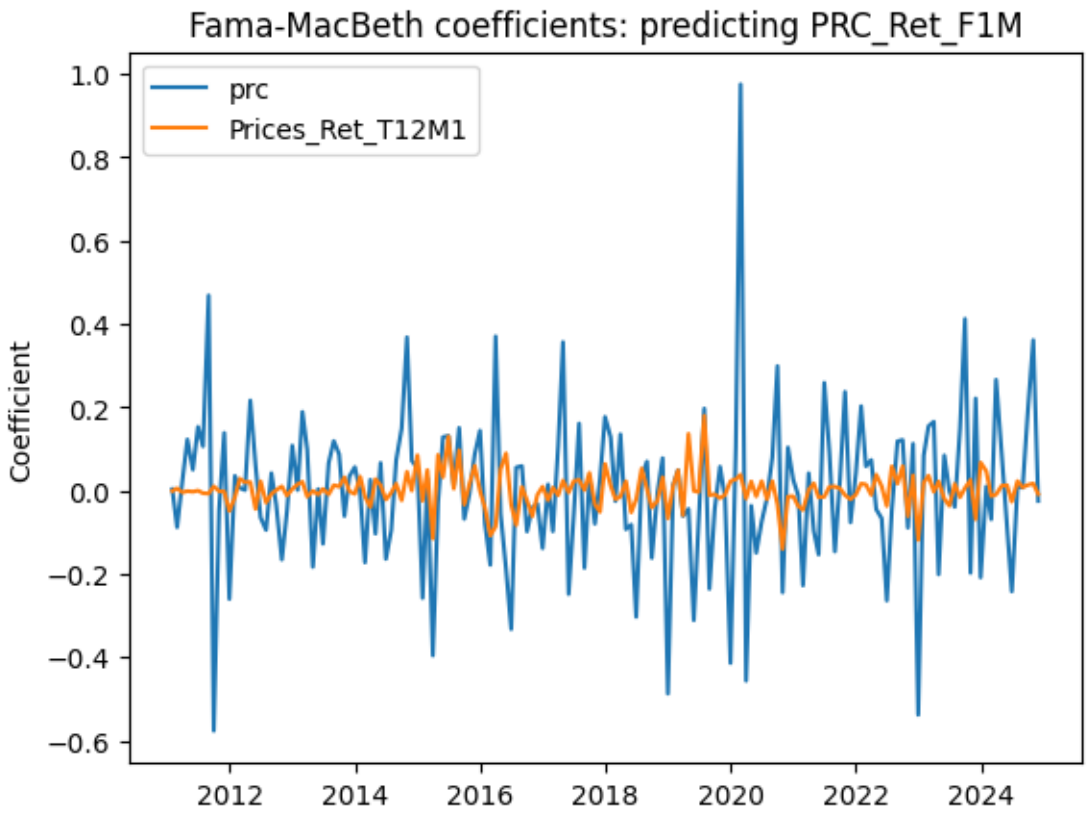
In the multivariate regressions, each specification combines two return-based variables that may capture different parts of return dynamics. For example, in the first regression, price_ret(t1) and Prices_Ret_T12M1 are designed to separate short term reversal from medium term momentum, so it is reasonable that they could have different signs.

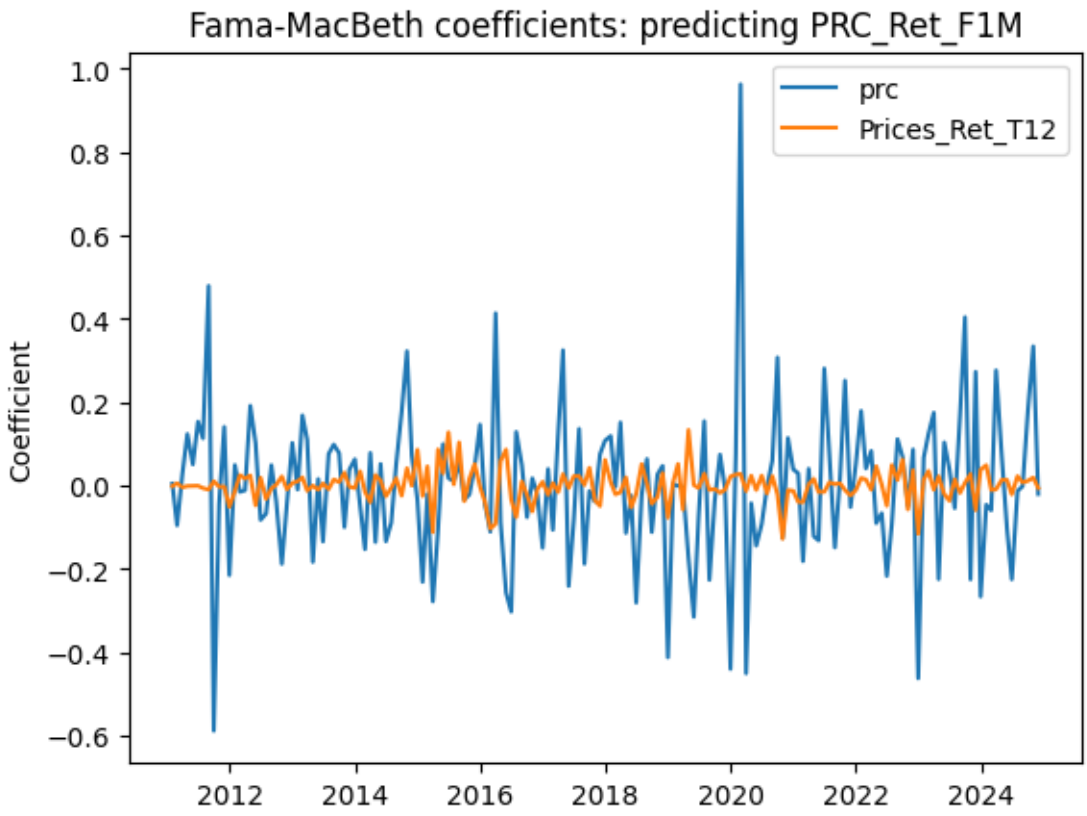
[25]:
```
fama_macbeth_regression(['price_ret(t1)', 'Prices_Ret_T12M1'], 'PRC_Ret_F1M')
fama_macbeth_regression(['prc', 'Prices_Ret_T12M1'], 'PRC_Ret_F1M')
fama_macbeth_regression(['prc', 'Prices_Ret_T12'], 'PRC_Ret_F1M')
fama_macbeth_regression(['prc', 'SR_Prices_Ret_T12M1'], 'PRC_Ret_F1M')
```

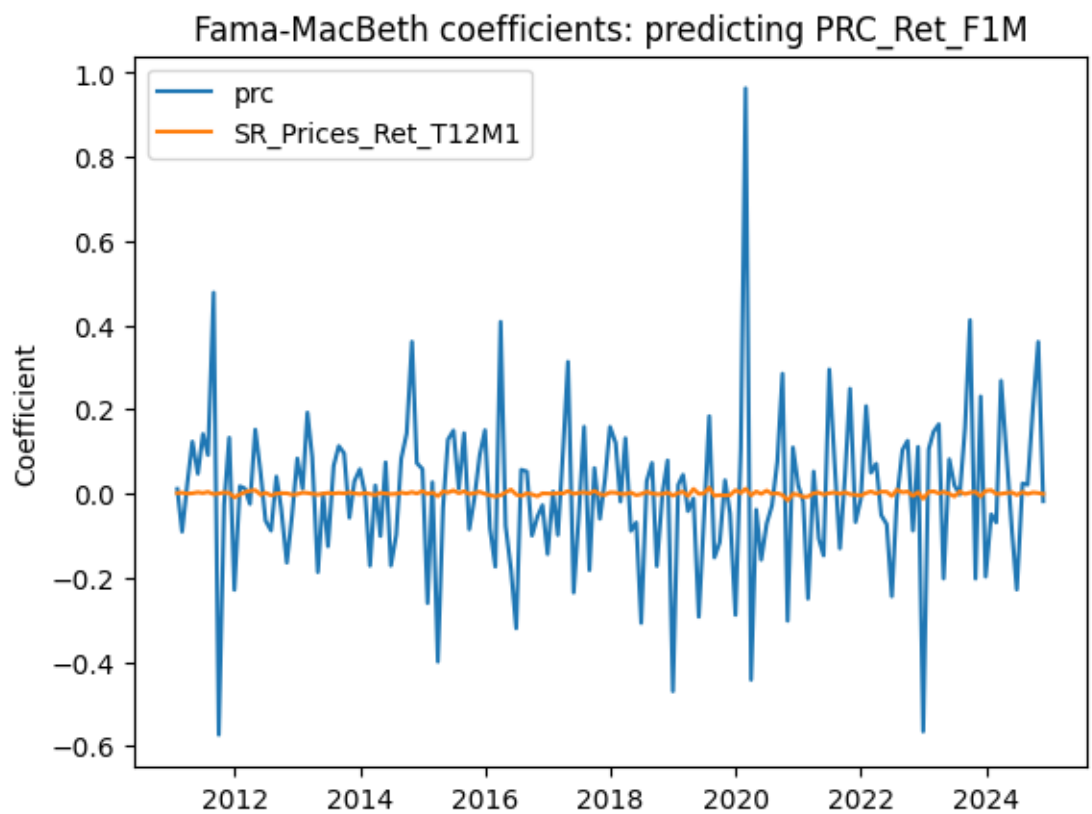
```
-------
Statistical significance t-stat of price_ret(t1) on PRC_Ret_F1M:  0.21793
Statistical significance t-stat of Prices_Ret_T12M1 on PRC_Ret_F1M:  0.51989
-------
Statistical significance t-stat of prc on PRC_Ret_F1M:  -0.00543
Statistical significance t-stat of Prices_Ret_T12M1 on PRC_Ret_F1M:  0.54248
-------
Statistical significance t-stat of prc on PRC_Ret_F1M:  -0.08588
Statistical significance t-stat of Prices_Ret_T12 on PRC_Ret_F1M:  0.24887
```

```
-------
Statistical significance t-stat of prc on PRC_Ret_F1M:  -0.01355
Statistical significance t-stat of SR_Prices_Ret_T12M1 on PRC_Ret_F1M:  0.69115
```




Fama-MacBeth coefficients: predicting PRC_Ret_F1M
prc
Prices_Ret_T12M1
Coefficient
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
2012
2014
2016
2018
2020
2022
2024


Fama-MacBeth coefficients: predicting PRC_Ret_F1M
prc
Prices_Ret_T12
Coefficient
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
2012
2014
2016
2018
2020
2022
2024

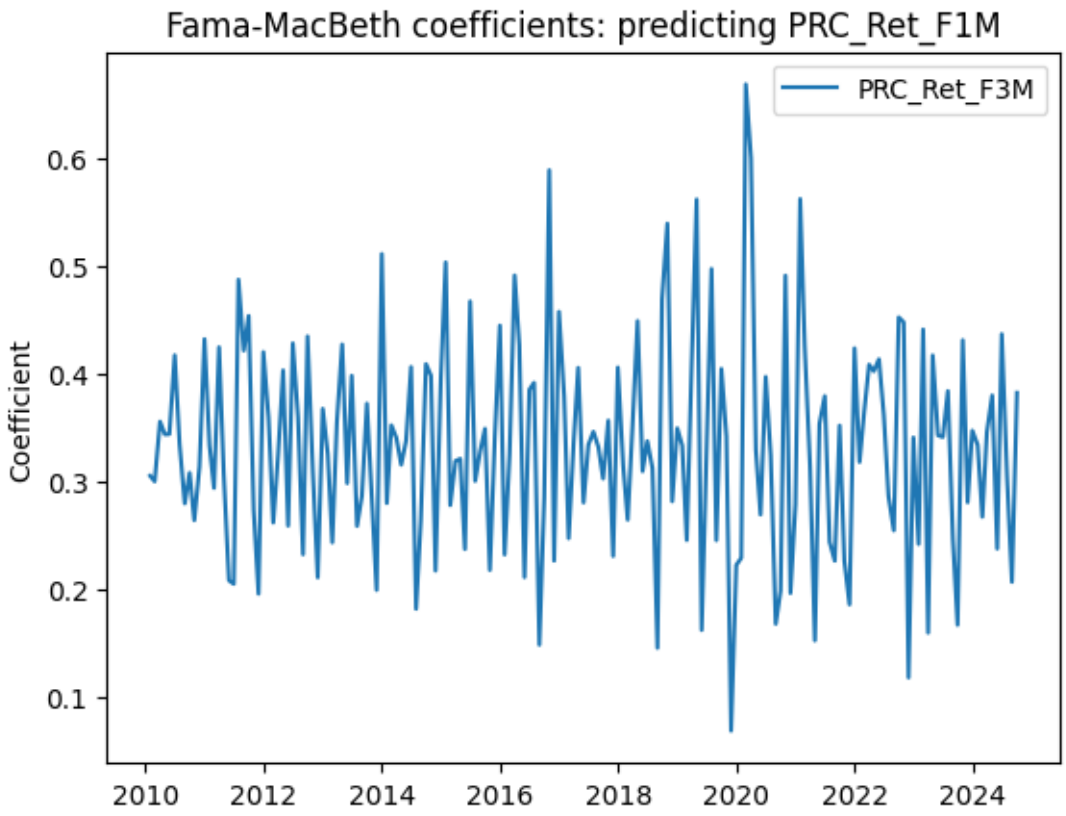


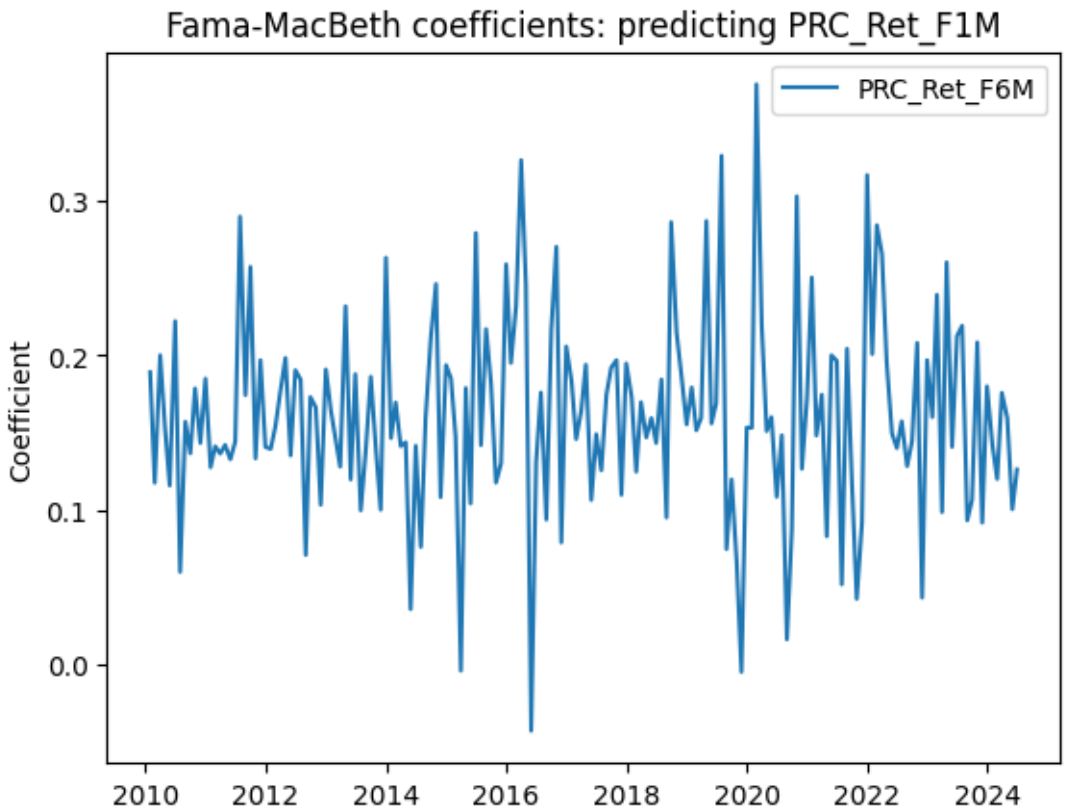
16 Question 9

The regressions use forward 3-month and 6-month returns to predict forward 1-month returns, which creates a mechanical overlap between the dependent and independent variables. For example, the forward 3-month return includes the next month's return as part of its calculation. This means the regression is partially using the same return to explain itself, creating a mechanical correlation rather than a true predictive relationship. As a result, the coefficients and t-statistics can appear artificially large even if there is no real predictive power. To fix this, the dependent and explanatory variables should be constructed so that their return windows do not overlap.

[26]:
```
fama_macbeth_regression(['PRC_Ret_F3M'], 'PRC_Ret_F1M')
fama_macbeth_regression(['PRC_Ret_F6M'], 'PRC_Ret_F1M')
```

```
-------
Statistical significance t-stat of PRC_Ret_F3M on PRC_Ret_F1M:  44.77789
-------
Statistical significance t-stat of PRC_Ret_F6M on PRC_Ret_F1M:  33.22388
```


Fama-MacBeth coefficients: predicting PRC_Ret_F1M
PRC_Ret_F3M
Coefficient
0.6
0.5
0.4
0.3
0.2
0.1
2010
2012
2014
2016
2018
2020
2022
2024



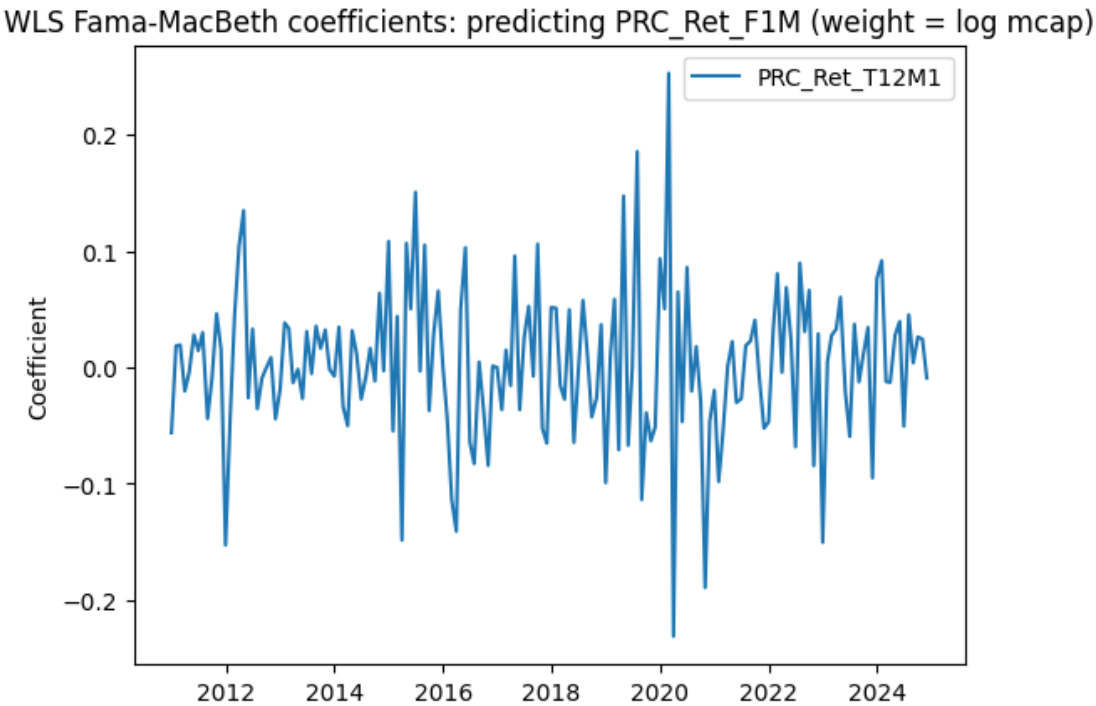
17 Question 10: Weighted Least Squares with log(mcap)

Re-run the Fama-MacBeth regression from 7(b), PRC_Ret_T12M1 predicting PRC_Ret_F1M, using **weighted least squares** where each observation is weighted by log(market cap).

[27]:
```
def fama_macbeth_regression_wls(indep, dep, weight_col='mcap',␣
↪months=all_months):
    """Fama-MacBeth regression using WLS with weights = log(weight_col)."""
    if not isinstance(indep, list):
        indep = [indep]

    coef = {}
    for m in months:
        DF_month = DF.loc[m]
        DF_month = DF_month[indep + [dep, weight_col]].dropna()
        # Drop any rows where mcap <= 0 (log undefined)
        DF_month = DF_month[DF_month[weight_col] > 0]
        if len(DF_month) > 0:
            X = sm.add_constant(DF_month[indep])
            Y = DF_month[dep]
```

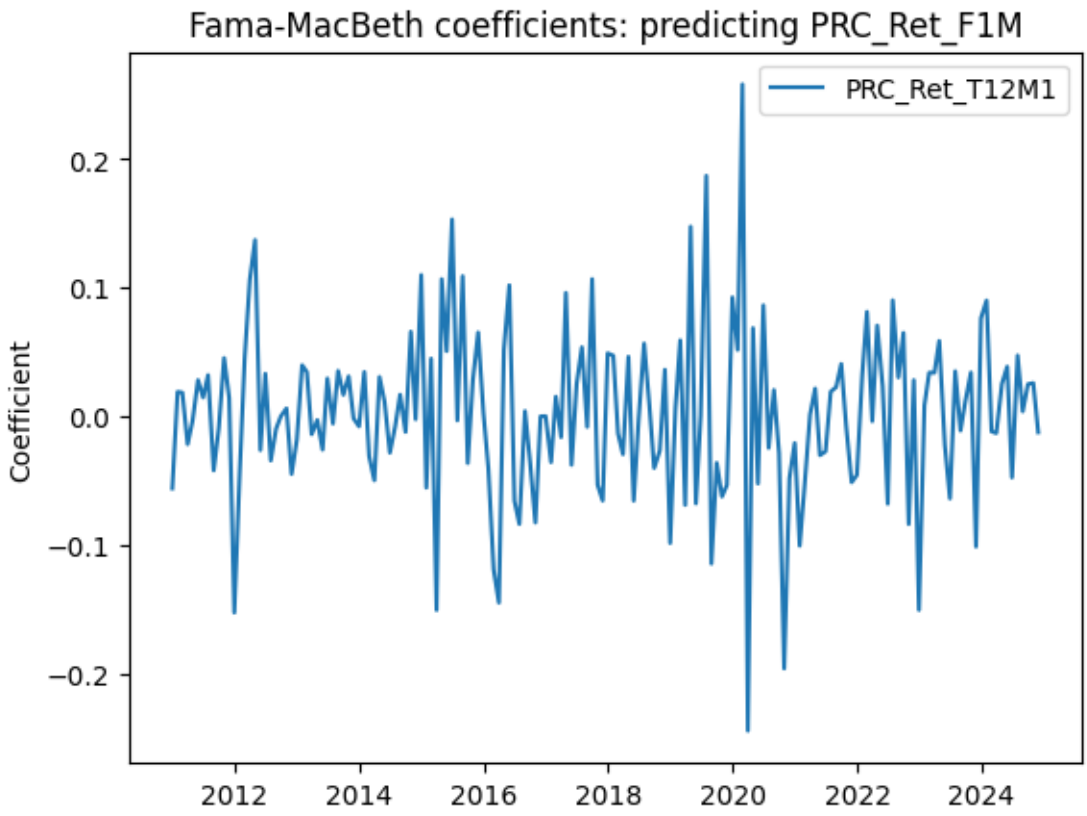
```
            W = np.log(DF_month[weight_col])
            model = sm.WLS(Y, X, weights=W).fit()
            coef[m] = dict(model.params)

    print("-------")
    print("WLS Fama-MacBeth (weights = log(mcap))")
    for i in indep:
        coefficients = [c[i] for c in coef.values()]
        coefficients_mean = np.mean(coefficients)
        coefficients_std = np.std(coefficients, ddof=1) / np.
↪sqrt(len(coefficients))
        print(f"Statistical significance t-stat of {i} on {dep}: ",␣
↪round(coefficients_mean / coefficients_std, 5))

    fig, ax = plt.subplots()
    for i in indep:
        coefficients = [c[i] for c in coef.values()]
        ax.plot(list(coef.keys()), coefficients, label=i)
    ax.set_title(f"WLS Fama-MacBeth coefficients: predicting {dep} (weight =␣
↪log mcap)")
    ax.set_ylabel("Coefficient")
    if len(indep) > 0:
        ax.legend()

fama_macbeth_regression_wls('PRC_Ret_T12M1', 'PRC_Ret_F1M')
```

```
-------
WLS Fama-MacBeth (weights = log(mcap))
Statistical significance t-stat of PRC_Ret_T12M1 on PRC_Ret_F1M:  0.48055
```



[28]:
```
# For comparison, re-print the original OLS result
print("Original OLS (equal-weighted) for comparison:")
fama_macbeth_regression('PRC_Ret_T12M1', 'PRC_Ret_F1M')
```

```
Original OLS (equal-weighted) for comparison:
-------
Statistical significance t-stat of PRC_Ret_T12M1 on PRC_Ret_F1M:  0.43952
```



17.0.1 Question 10 – Analysis

Comparison of results:

The equal-weighted OLS Fama-MacBeth regression from Q7(b) produced a t-statistic of approximately 0.44, which was already not statistically significant. The WLS regression with log(mcap) weights produces a t-statistic that is similarly weak (and may differ slightly in magnitude or sign), confirming that the momentum signal PRC_Ret_T12M1 does not reliably predict next-month returns regardless of weighting scheme.

Why weight by market cap at all?

Equal-weighted OLS treats every stock identically: a $600M small-cap gets the same influence as a $300B mega-cap. In practice, institutional portfolios are much closer to value-weighted. Large-cap stocks receive the bulk of invested capital, are more liquid, and have lower trading costs. An equal-weighted regression overrepresents small-cap behavior, which tends to be noisier, less liquid, and harder to trade. By weighting by market cap, we test whether the signal works where most investable capital actually lives.

Why log(mcap) specifically, rather than raw mcap?

Raw market cap in the S&P 500 is extremely right-skewed: the largest companies (Apple, Microsoft, etc.) have market caps of ~$3T while the smallest constituents are around $600M. If we used raw mcap as weights, a handful of mega-cap stocks would completely dominate the regression and

effectively determine the coefficient by themselves. Log(mcap) compresses this scale: it still gives more weight to larger stocks, but in a much more balanced way. For example, a stock with 100x the market cap of another would get roughly 2.0x the weight under log(mcap) rather than 100x the weight. This makes log(mcap) a sensible compromise between equal-weighting (which ignores size entirely) and raw mcap-weighting (which is dominated by a few mega-caps).

What issues should we be aware of?

1. **Heteroskedasticity:** Small-cap returns tend to be more volatile than large-cap returns. In an equal-weighted OLS regression, these noisier observations receive the same weight as more stable large-cap observations, potentially distorting inference. WLS with log(mcap) partially addresses this by downweighting smaller, noisier stocks.
2. **Investability and capacity:** A signal that only works for small-cap stocks may not be implementable at institutional scale: you cannot deploy billions of dollars into illiquid small-caps without massive market impact. If the momentum signal weakens under log(mcap) weighting, it suggests the signal is driven by hard-to-trade small stocks and is therefore less practically useful.
3. **Microstructure effects:** Small stocks have wider bid-ask spreads, less analyst coverage, and greater information asymmetry. Momentum effects can appear artificially stronger in small-caps purely due to these frictions (e.g., stale prices, slower information incorporation) rather than a genuine return-predictive relationship.
4. **No look-ahead bias concern:** The weighting uses contemporaneous mcap, which is known at the time of portfolio formation, so this weighting scheme does not introduce any look-ahead bias.

18 Question 11: First Principles Reflection

Why this homework?

This homework is designed to walk us through the full lifecycle of quantitative investment research, from raw data to signal construction to statistical testing to critical interpretation. Rather than presenting momentum as a textbook "fact," it forces us to confront what happens when we actually try to build and evaluate a momentum strategy from scratch on real data.

Signal construction matters enormously. We built many variations of what is loosely called "momentum": total return vs price return, 12-month trailing vs 11-month (excluding the most recent month), the single return from exactly 12 months ago, volatility-adjusted momentum, and more. These are not just cosmetic differences; each captures a subtly different economic hypothesis. The classic Jegadeesh-Titman [-12, -1] window (PRC_Ret_T12M1) deliberately separates medium-term momentum from short-term reversal, because the most recent month's return tends to reverse (a microstructure effect) while the prior 11 months may exhibit continuation. The single-month return from 12 months ago (PRC_Ret_T12_1M) captures potential annual seasonality effects. The volatility-adjusted Sharpe ratio version (SR_Prices_Ret_T12M1) tests whether risk-adjusted momentum is more predictive than raw momentum. The lesson is that small choices in signal definition can change results dramatically, and a quantitative researcher must be intentional and precise about what economic hypothesis each signal encodes.

Total return vs price return. We learned early on (Q2) that the `prc` column is total return

(including dividends) while `price` is the bid-ask midpoint. Using price return ignores dividends, which matters for dividend-paying stocks. Total return is the economically correct measure for evaluating investment performance. Carrying both through the analysis forced us to understand this distinction and see how it affects results.

The dependent variable matters just as much. We tested F1M, F3M, and F6M forward returns. Q9 delivered a critical lesson: using overlapping forward windows (e.g., regressing F3M on F1M) creates mechanical correlations that produce spuriously significant results. This is a real trap in empirical finance; overlapping return windows are a common source of false discoveries. Choosing the right evaluation horizon is a genuine research design decision, not a trivial detail.

Statistical significance vs economic significance. A single cross-section (Q7a, November 2019) showed a strongly significant coefficient (t = -5.15), but the full-sample Fama-MacBeth t-statistic (Q7b) was only 0.44. This demonstrates that cross-sectional relationships can be strong in any given month but completely unstable over time. A strategy based on a signal with a t-statistic near zero is not reliably profitable. The time-series plot of coefficients, fluctuating around zero and changing sign frequently, makes this instability visually clear. This teaches the essential distinction between finding a pattern in one snapshot vs having a robust, repeatable signal.

Equal-weighted vs value-weighted analysis (Q10). By re-running the regression with log(mcap) weights, we tested whether the signal works where capital actually gets deployed. This addresses investability: a signal that only works for tiny stocks is academically interesting but practically useless. It also addresses heteroskedasticity, since small-cap returns are noisier. This kind of robustness check is fundamental to serious quantitative research.

Momentum may not work in recent data. The classic momentum literature (Jegadeesh and Titman, 1993) used data from 1965-1989 and found strong results. Our sample (2010-2024) shows weak, insignificant momentum. This is perhaps the most important lesson of the homework: published factors can decay over time, whether due to arbitrage by informed traders, changes in market microstructure (the rise of high-frequency trading, ETFs, and passive investing), or regime shifts. The "momentum crash" of 2009 and the subsequent decade of changed market dynamics may explain why the signal has weakened. A quantitative researcher must maintain healthy skepticism about any historical backtest, no matter how celebrated in the academic literature.

The Fama-MacBeth procedure as a foundational tool. The homework taught us the mechanics of Fama-MacBeth regression: run monthly cross-sectional regressions, collect the time series of coefficients, and perform inference from the time-series mean and standard deviation. This procedure is foundational in empirical asset pricing; the monthly regression coefficient is effectively the return to a long-short factor portfolio, and the time-series t-statistic tells us whether that factor reliably earns a premium. Understanding this connection between regression coefficients and factor returns is essential for any quantitative researcher.

Data quality comes first. The homework began with data exploration (Q1-Q5) before any signal construction or regression. We investigated missing values, understood why they occurred (M&A events, first observations), examined index composition dynamics, and visualized market cap distributions. This is not busywork; it is the most important step. Garbage in, garbage out. Understanding your data, its quirks, its coverage, its potential biases, is what separates rigorous research from spurious findings.

Overall takeaway. This homework taught us that quantitative investment research is not about finding one magic signal. It is about the disciplined process of understanding data, constructing

signals with clear economic hypotheses, testing those signals rigorously with appropriate statistical methods, checking robustness across weighting schemes and time periods, and maintaining a healthy skepticism about any result, especially one that looks too good to be true.